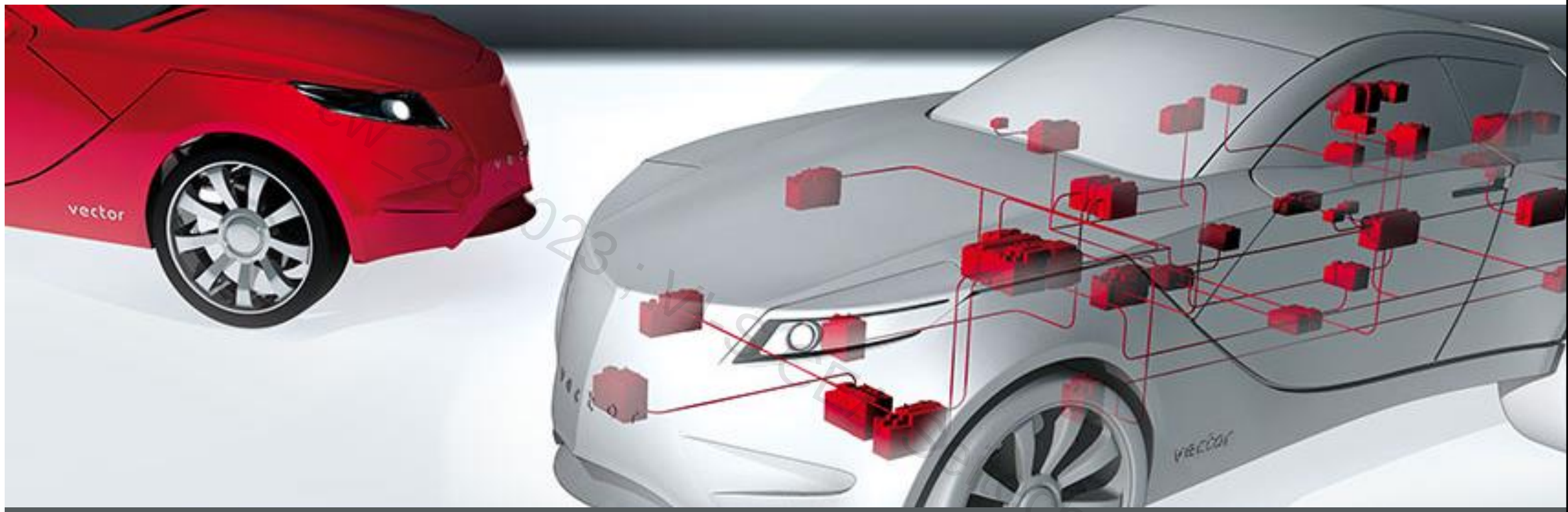
VECTOR



AUTOSAR in Practice

I/O

Agenda

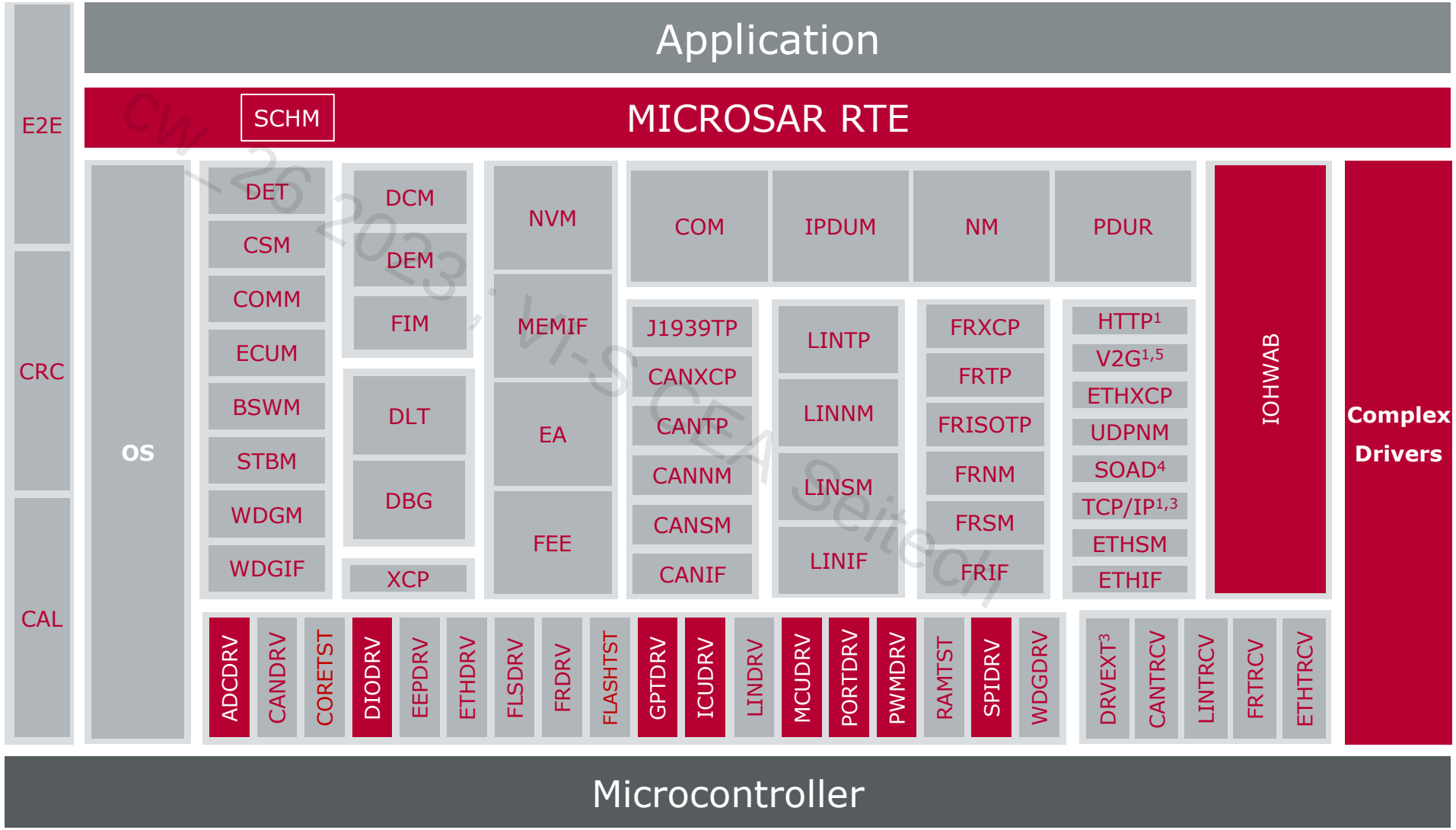
- **Architecture of I/O**
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Elements for Input / Output in AUTOSAR

Affected BSW Modules

- I/O Hardware Abstraction
- ICU Driver
- OCU Driver
- PWM Driver
- ADC Driver
- DIO Driver
- PORT Driver
- GPT Driver
- SPI Driver



CW 26 2023 - VI-SCEA Seitech

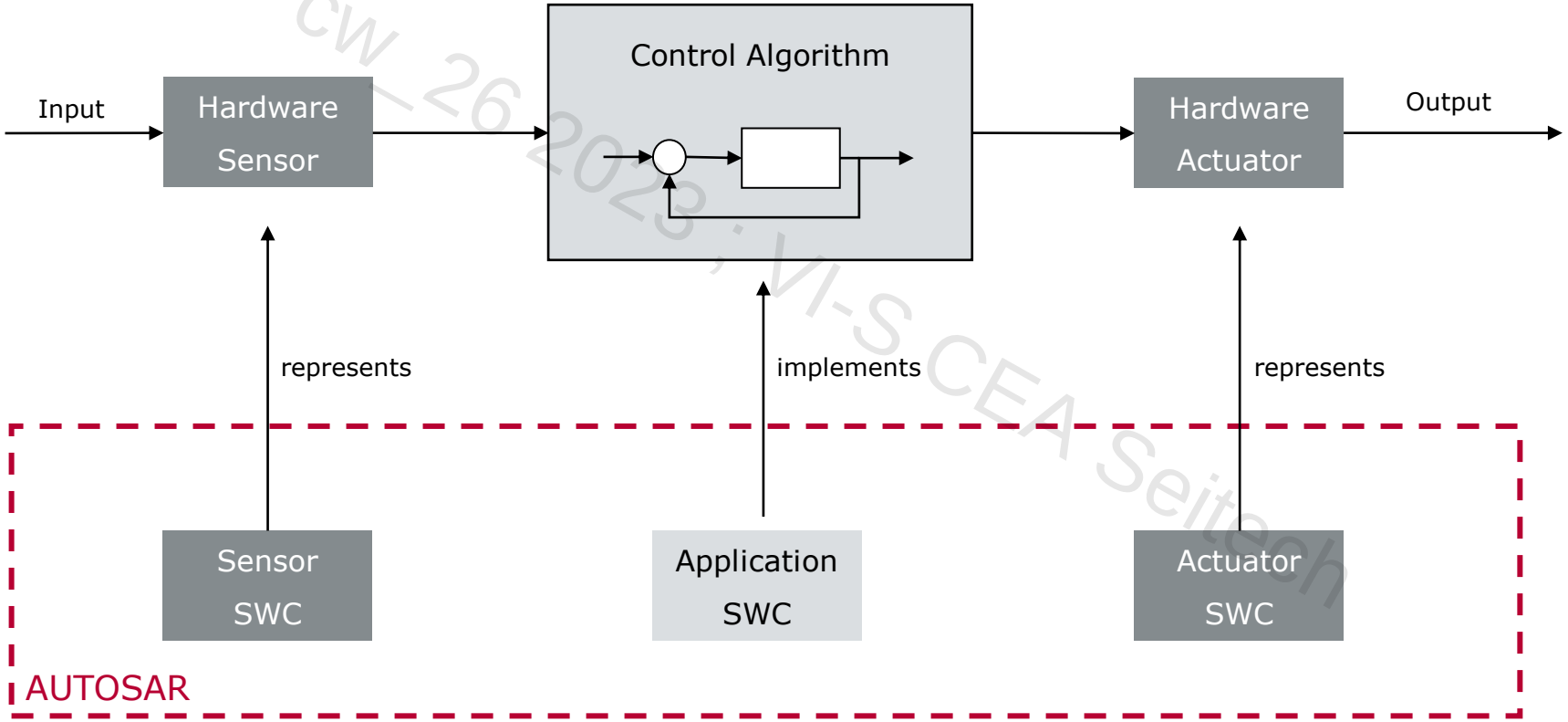
Elements for Input / Output in AUTOSAR

We will focus on the Microcontroller Abstraction Layer (MCAL), which is provided by the hardware vendor.

- The MCAL provides a standardized access to the HW of your microcontroller to the upper layers
- No signal state or pin state conversions are done by the MCAL

cw_26_2023 ; VI-S CEA Seitech

Generic model of an application



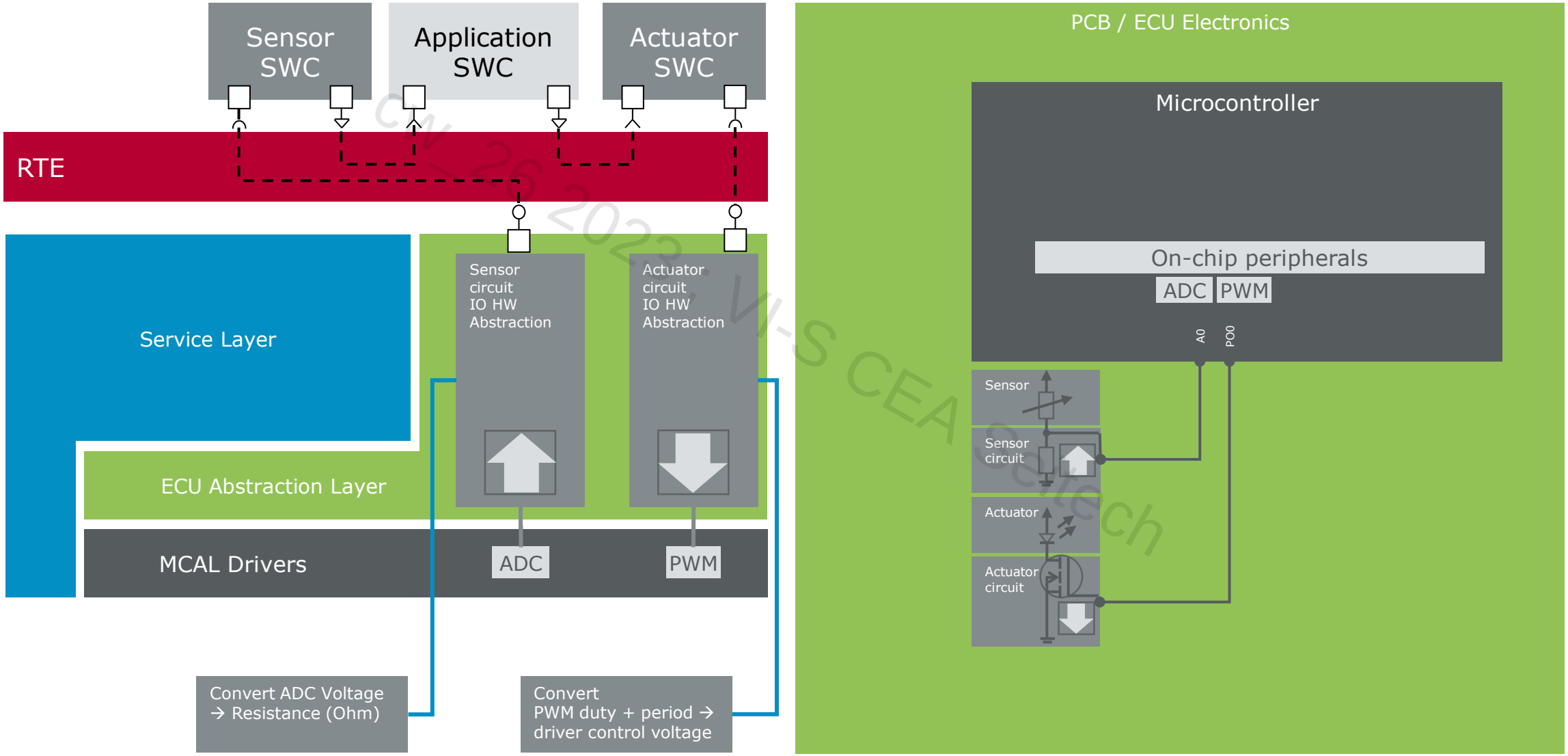
cw_26_2023 ; VI-S CEA Seitech

Generic model of an application

A main element of automotive applications are control algorithms which perform a function like fuel injection or temperature control. Control algorithms calculate output values based on certain input values. If the inputs and outputs are located on the same ECU as the control algorithm, then input/output peripherals of the Microcontroller or ECU are used. Usually sensor values have to be read and actuators have to be controlled. The following slides show how input/output control can be designed in AUTOSAR.

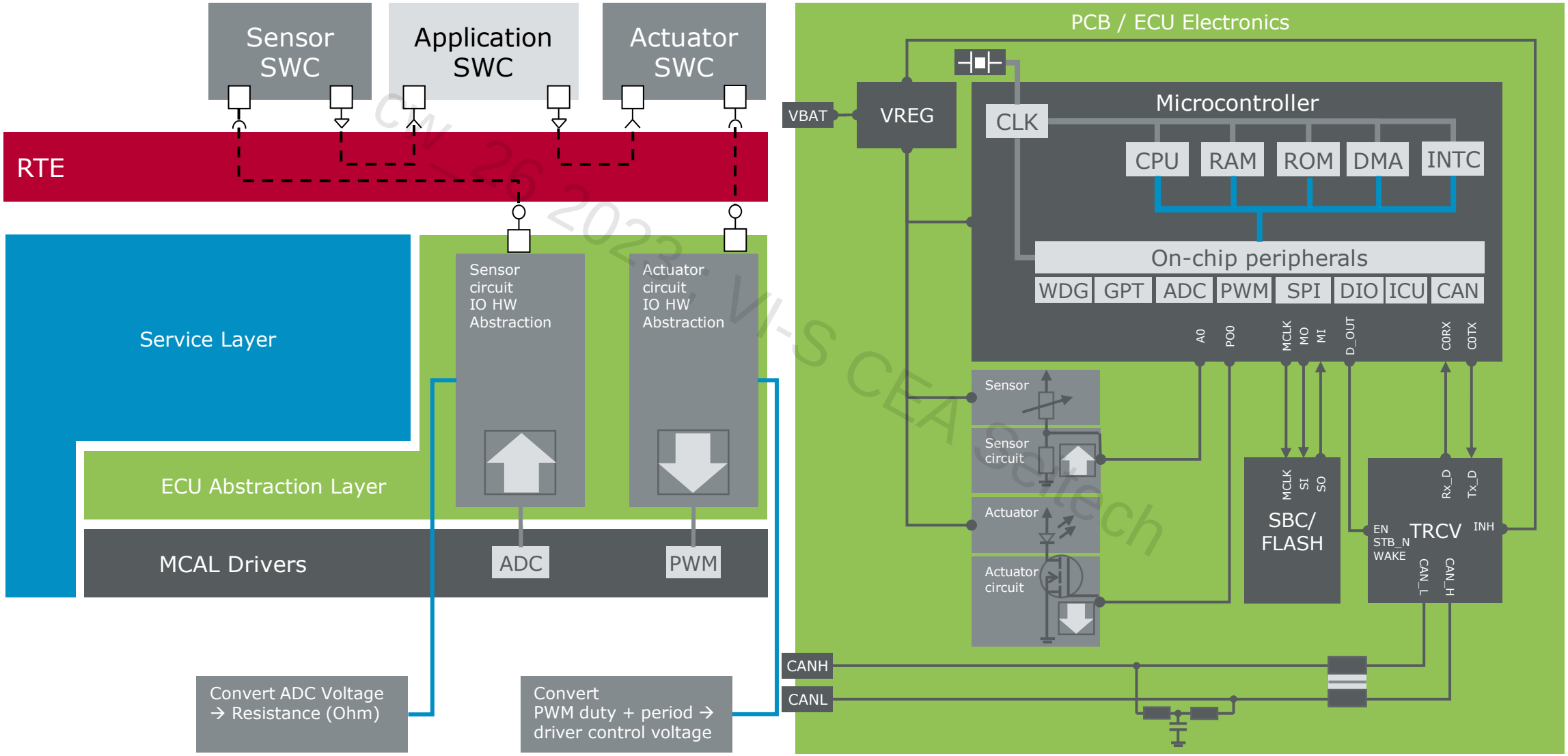
CV_26_2023 ; VI-S CEA Seitech

The ECU Abstraction Layer



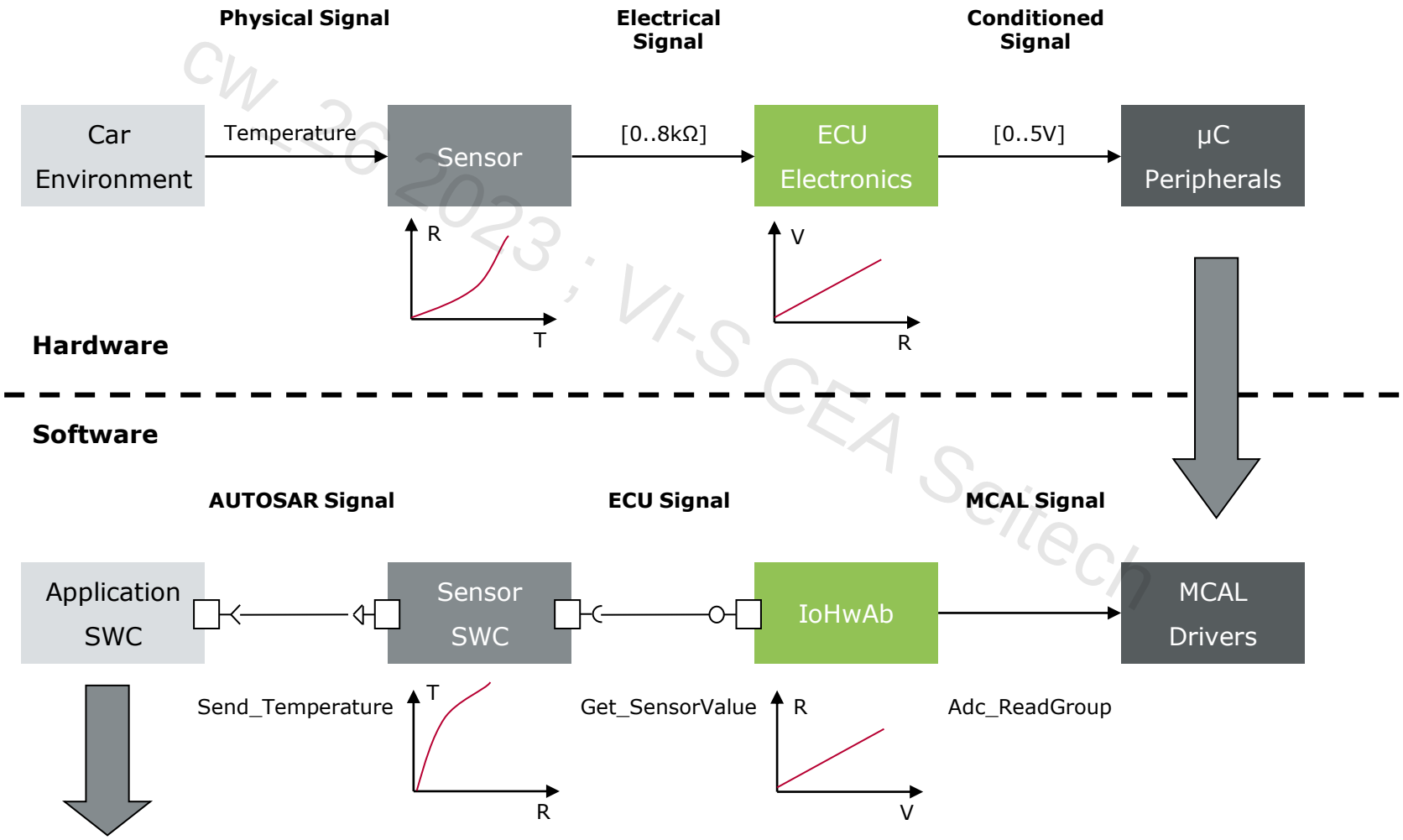
CW 26 2022 VI-S CEA Setech

The ECU Abstraction Layer



CW 26 2022 VI-S CEA Setech

Sensor Signal Path



cw 28 2023 ; VI-SCEA Scitech

Sensor Signal Path

Sensor: Converts a physical or environmental signal into an electrical signal

ECU Electronics: Electronic circuit which converts/conditions the sensor's electrical signal into an input signal suitable for the microcontroller (e.g. 5V, 3.3 V)

µC Peripherals: Analog to Digital Converter, Capture Compare Unit, SPI

MCAL Drivers: AUTOSAR standardized peripheral software drivers

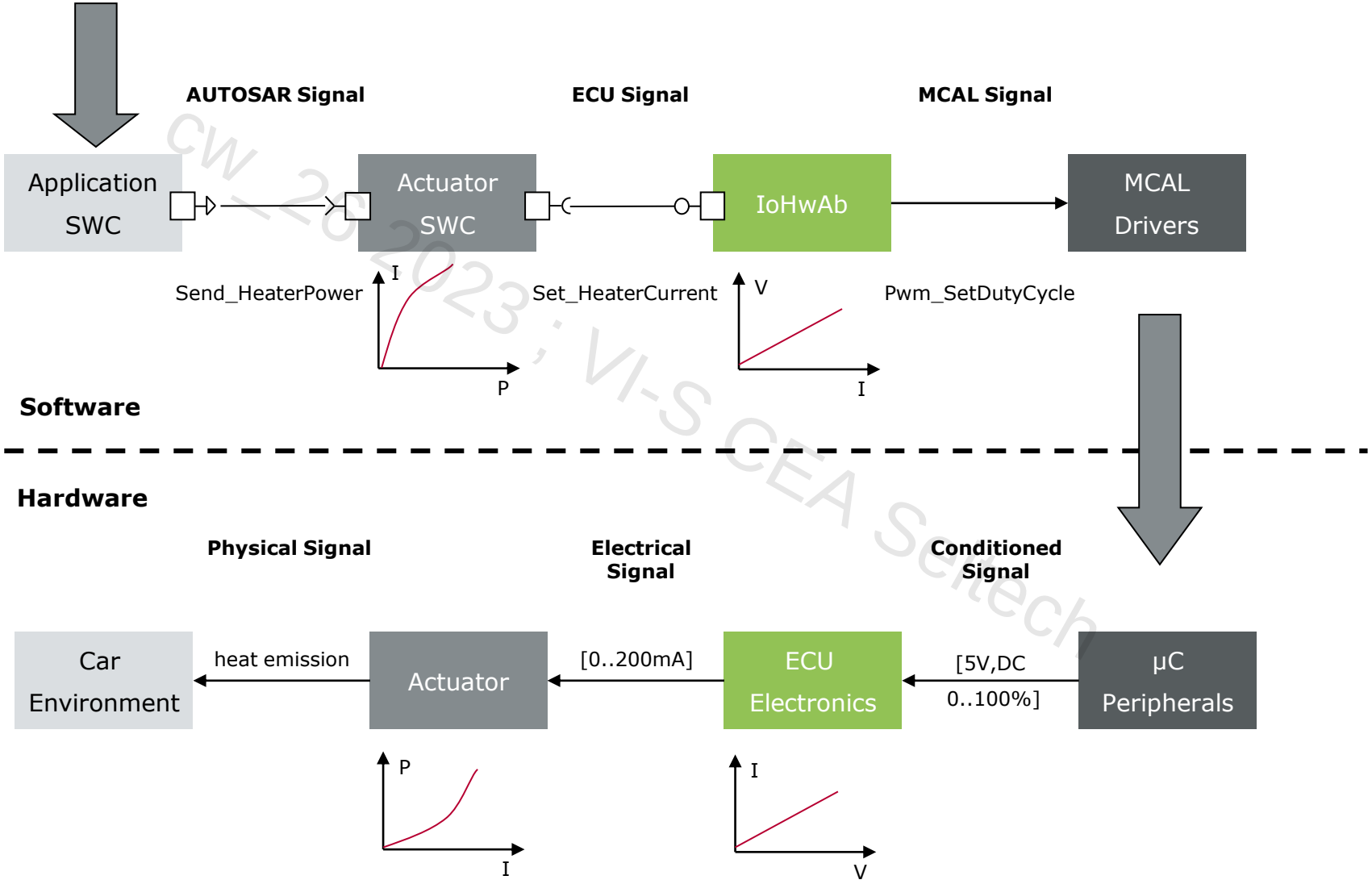
I/O Hardware Abstraction: Does signal conditioning/debouncing/filtering. Converts from hardware-specific raw values into physical values

Sensor SWC: SW representation of hardware sensor. Converts electrical input values into application signals. Has knowledge of sensor hardware characteristics

Application SWC: Implements algorithm for control of sensors and actuators

CW 20 2026 : V1-9 CEA Seitech

Actuator Signal Path



cw 28 2023 ; VI-S CEA Sentech

Actuator Signal Path

Application SWC: Implements algorithm for control of sensors and actuators

Actuator SWC: SW representation of hardware actuator. Converts the application signals/commands into hardware values. Has knowledge of actuator hardware characteristics

I/O Hardware Abstraction: Does signal conditioning/debouncing/filtering. Converts from hardware specific raw values into physical values

MCAL Drivers: AUTOSAR standardized peripheral software drivers

µC Peripherals: PWM Module, Digital IO, SPI

ECU Electronics: An electronic circuit which converts/conditions the sensor's electrical signal into an input signal for the microcontroller (e.g. 5V, 3.3 V)

Actuator: Converts an electrical signal into a physical or environmental signal

cw 26 2023 · VI-S CEA Schlech

Agenda

- Architecture of I/O
- **I/O Drivers – General Characteristics**
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

- Basic driver access

- No data conversion -> plain reading of register values
  - PORT: No level inversion
  - ADC: No value conversion
  - PWM/ICU: No time/frequency conversion

- Time unit of register values is ticks
  - Actual time values depend on the MCU settings
  - Conversion from ticks to time is not done
    - Has to take place in the I/O Hardware Abstraction

cw 26 2023 ; VI-S CEA Seitech

No time conversion for PWM or GPT. Basic tick value is based on the peripheral input clock domain.

- Drivers make use of dedicated hardware functions
  - E.g., PWM, Capture Compare Unit

- Common concept
  - Channel -> I/O pin, ADC/PWM channel
  - Channel Group -> group of pins, group of ADC channels
  - Port -> a port register (e.g. 8-bit port)

cw_26_2023, VI-S CEA Seitech

MCALs work with the dedicated HW functions and peripherals!!!

Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- ▶ **PORT Driver**
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

VECTOR

Characteristics

- Configuration of ports
  - Mode (GPIO/alternative mode)
  - Direction (input/output)
  - Direction changeable during runtime (yes/no)
  - Initial pin level (low/high)
- Optional
  - Slew rate control
  - Activation of internal pull ups
  - Input thresholds
  - Pin driver mode (push-pull/open drain)
  - Type of read-back support (pin level, output register value)

CW_26 2023 ; VI-S CEA Seitzer

PORT Driver API:

`Port_Init`**:** Initialize port data register of a specific port pin

`Port_SetPinDirection`**:** Enables setting of the port pin direction during runtime

`Port_RefreshPortDirection`**:** Refresh direction of non-dynamically configurable port pins

`Port_SetPinMode`**:** Change the pin mode (GPIO or alternative function) during runtime

Port Driver needed to configure port functions like e.g. CAN_RX

→ This is not done by the CAN driver

→ Port Driver is HW dependent and may always look a bit different between platforms

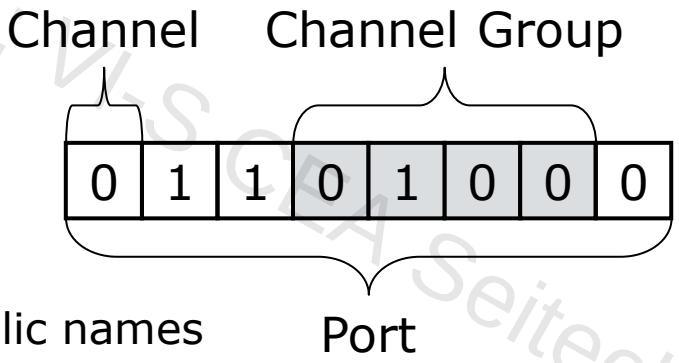
Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- ▶ **DIO Driver**
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Characteristics

- Abstracts the access to digital I/O pins (channels)
- Allows grouping of port pins (channel groups)
- Provides read/write access of the General Purpose digital I/O for
  - Channels
  - Channel groups
  - Ports



- Access to elements by user-defined symbolic names

cw_26 2023 , V-S GEA Seitech

Characteristics

Used by the BSW, e.g. by the Transceiver Drivers:

- Inside the Transceiver Driver an reference to a DIO channel is provided

Definition of Channel, Channel Group, Port

Channel -> a single port pin

Channel group -> a combination of **adjacent** DIO channels

Port -> a port register (e.g. 8-bit port)

DIO Driver API:

`Dio_ReadPort:` Reading of a complete port register

`Dio_WritePort:` Writing of a complete port register

`Dio_ReadChannelGroup:` Reading of a channel group

`Dio_WriteChannelGroup:` Writing of a channel group

`Dio_ReadChannel:` Reading of a single channel (Port-Pin)

`Dio_WriteChannel:` Writing of a single channel (Port-Pin)

`Dio_FlipChannel :` Reading of an output channel, inversion and writing of this value

CW 26 2023, VI-S CEA Seitech

Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- **PWM Driver (for reference)**
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Characteristics

- Generation of Pulse-Width-Modulated (PWM) Signals using specific PWM hardware functions
- PWM configuration parameters for each channel
  - Assigned HW channel
  - Period
  - Duty Cycle
  - Polarity (high/low)
  - Idle State (high/low)
  - Channel Class
    - Fixed period
    - Fixed period, shifted (if supported by hardware)
    - Variable period

SW_26 2023 ; VI-S CEA Seitech

No support for software PWM. Software PWM can be implemented in the I/O Hardware Abstraction using DIO Channels.

PWM module needs dedicated hardware peripheral.

Characteristics

- Optional configuration parameters
  - Phase shift
  - Reference channel for phase shift
  - Microcontroller specific channel properties

- Duty cycle -> 16 bit value
  - 0x0000 -> 0%
  - 0x8000 -> 100%
- `Pwm_Main_PowerTransitionManager`
  - Used in case of asynchronous power management, otherwise no periodic main function.

CW_26 2023 ; VI-S CEA Seitech

Characteristics

PWM Driver API

`Pwm_Init`: Initializes all internal variables and resources and starts all PWM channels with the configured default values; All notifications are disabled

`Pwm_DeInit`: De-initializes the PWM module; disables all PWM interrupts and notifications; Sets the PWM output signals to the configured idle states

`Pwm_SetDutyCycle`: Sets the duty cycle (Min = 0x0000, Max = 0x8000)

`Pwm_SetPeriodAndDuty`: Sets the period and duty cycle

`Pwm_SetOutputToIdle`: Sets the PWM output signals to the idle state

`Pwm_GetOutputState`: Reads the internal state of the PWM output signal

`Pwm_EnableNotification`: Enables the PWM signal edge notification (PWM_RISING_EDGE, PWM_FALLING_EDGE, PWM_BOTH_EDGES)

`Pwm_DisableNotification`: Disables the signal edge notification

`Pwm_SetPowerState`: Set the power mode of the PWM HW Unit. Configures the Pwm module so that it enters the already prepared power state, chosen between a predefined set of configured ones.

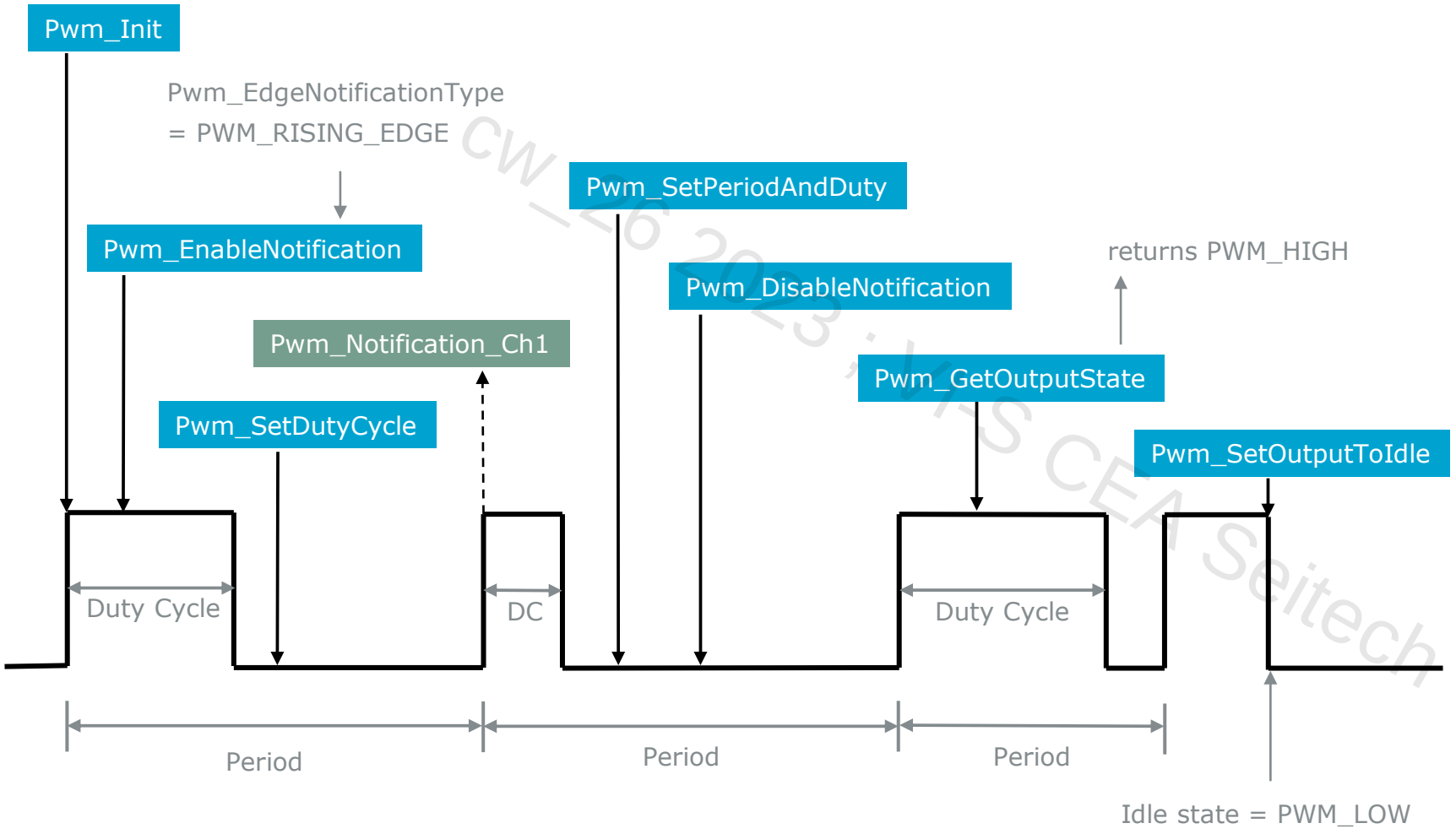
`Pwm_GetCurrentPowerState`: Get the current power mode of the PWM HW Unit

`Pwm_GetTargetPowerState`: Target power mode of the PWM HW Unit. Returns the requested power state of the HW unit. This shall coincide with the current power state if no transition is ongoing. The API is considered to always succeed except in case of HW failures.

`Pwm_PreparePowerState`: This API starts the needed process to allow the PWM HW module to enter the requested power state. This API initiates all actions needed to enable a HW module to enter the target power state. The possibility to operate the peripheral depends on the power state and the HW features. These properties should be known to the integrator and the decision whether to use the peripheral or not is in his responsibility.

cw_26_2023 ; VUS CEA Seitech

Example



CW_26_2023_VRS CEA Seitech

Example

1) Pwm_Init is called. PWM starts with default duty cycle and period. Polarity = PWM_HIGH. Idle state = PWM_LOW
2) Notification (rising edge) is enabled. Notification Callback is called on next rising edge. Callback API: Pwm_Notification_<Channel No>. In Example "Ch1" is used (user-defined name)
3) Duty Cycle is set to a smaller value. Change takes effect at the end of the period
4) Period and Duty Cycle are changed. Change takes effect at the end of the period
5) Notification is disabled
6) Output state is checked using Pwm_GetOutputState. PWM_HIGH is returned
7) PWM is set to idle state => LOW (Interrupts still active)

Not in picture:

8) To restart the PWM signal generation: Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty has to be called
9) Pwm_DeInit: all interrupts disabled, all Notifications disabled, PWM output set to configured idle state

CW_26_2023 : VI-S CEA Seitech

Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- **ICU Driver (for reference)**
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Characteristics

- Input Capture Unit (ICU) is used for
  - Edge Count
  - Signal Edge Detection / Notification
  - Time stamping
  - Signal Measurement

CW_26 2023 : VI S CEA Seitech

Used as well to generate an interrupt based on the detection of some signal edge:

Used for wakeup detection (either from CAN Transceiver or dedicated wakeup input)

Can be used for bringing MCU back from sleep mode

General API Functions

`Icu_Init`

Initializes the relevant registers of the configured hardware

Disables all notifications

Disables the wakeup capability of all channels

Sets status to ICU_IDLE

Sets module mode to ICU_MODE_NORMAL

`Icu_DeInit`

Sets the state of the peripheral to same state as after power-on reset

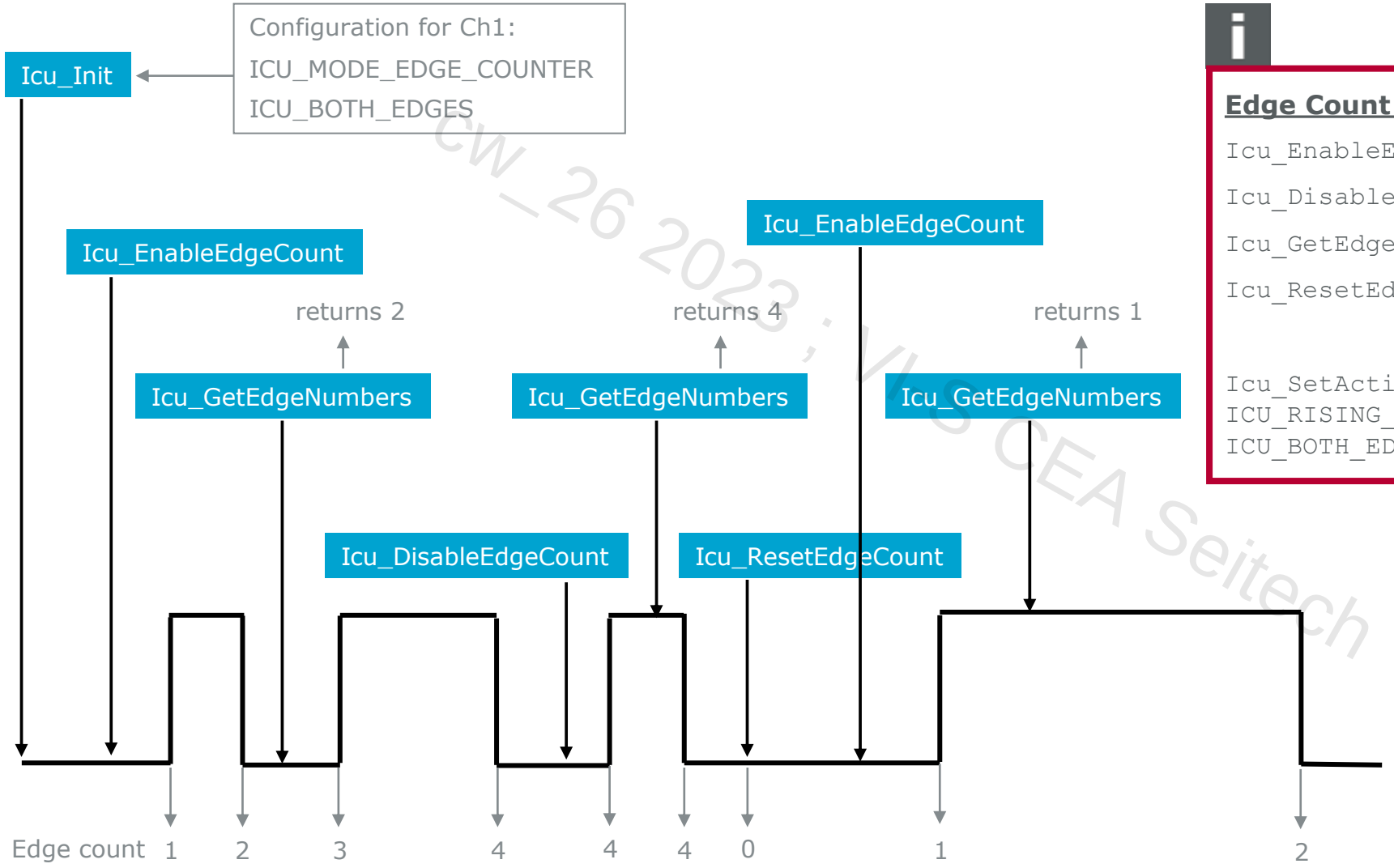
Wakeup Functionality

- ICU Driver can be used as a Wakeup Source

- Wakeup API
  - `Icu_SetMode`
  - `Icu_EnableWakeup`
  - `Icu_DisableWakeup`
- From ISR or cyclically by EcuM:
  - `EcuM_CheckWakeup` → `Icu_CheckWakeup` → `EcuM_SetWakeupEvent`

cw_26 2023 ; VI-S CEA Seitech

Edge Count - Example



Edge Count API:

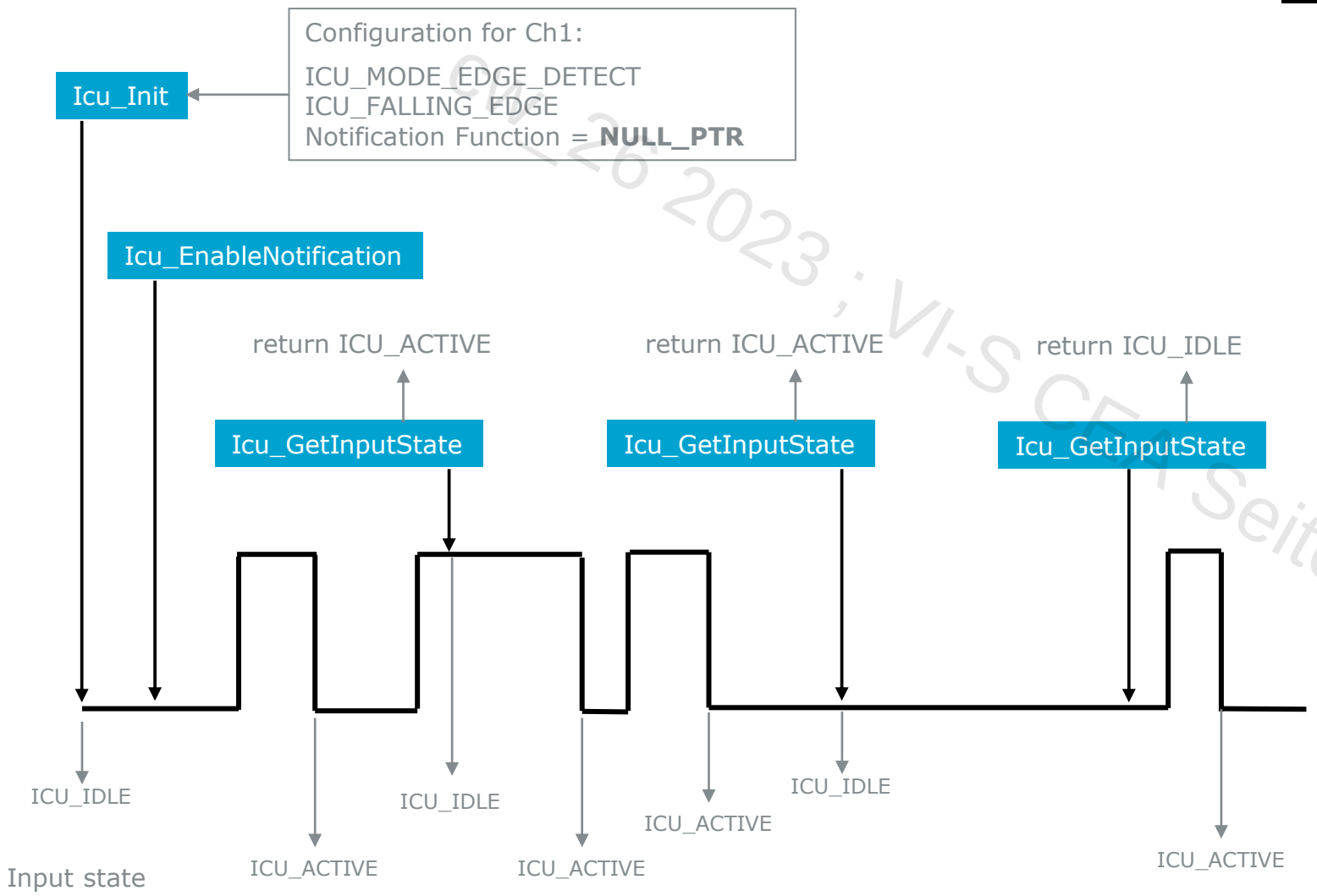
```
Icu_EnableEdgeCount
Icu_DisableEdgeCount
Icu_GetEdgeNumbers
Icu_ResetEdgeCount

Icu_SetActivationCondition:
ICU_RISING_EDGE, ICU_FALLING_EDGE,
ICU_BOTH_EDGES,
```

cw_26 2023 ; VLS CEA Seitech

Signal Edge Detection – Example 1

Polling



Oct 26 2023 ; VI-S CEA Seitech

Signal Edge Detection – Example 1

ICU_ACTIVE is returned if a falling edge was detected before the latest Icu_GetInputState call

Note: No notification raised because Notification function is defined as NULL_PTR

Signal Edge Detection API

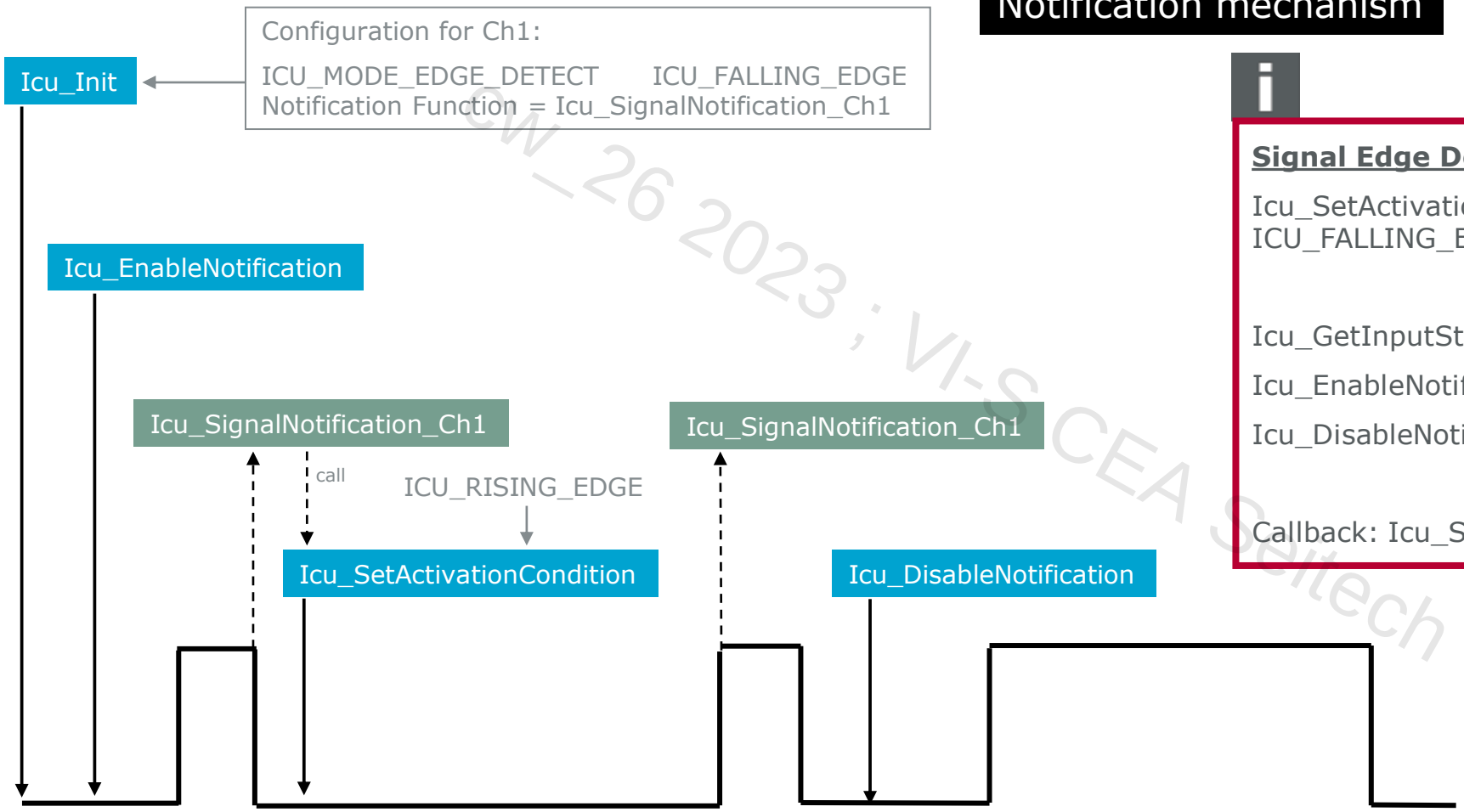
`Icu_SetActivationCondition: ICU_RISING_EDGE, ICU_FALLING_EDGE, ICU_BOTH_EDGES`

```
Icu_GetInputState
Icu_EnableNotification
Icu_DisableNotification
Icu_EnableEdgeDetection
Icu_DisableEdgeDetection
```

Callback: `Icu_SignalNotification_<Channel>`

cw_26_2023 ; VI-S CEA Seitech

Signal Edge Detection – Example 2



Notification mechanism

Signal Edge Detection API

Icu_SetActivationCondition: ICU_RISING_EDGE, ICU_FALLING_EDGE, ICU_BOTH_EDGES

Icu_GetInputState

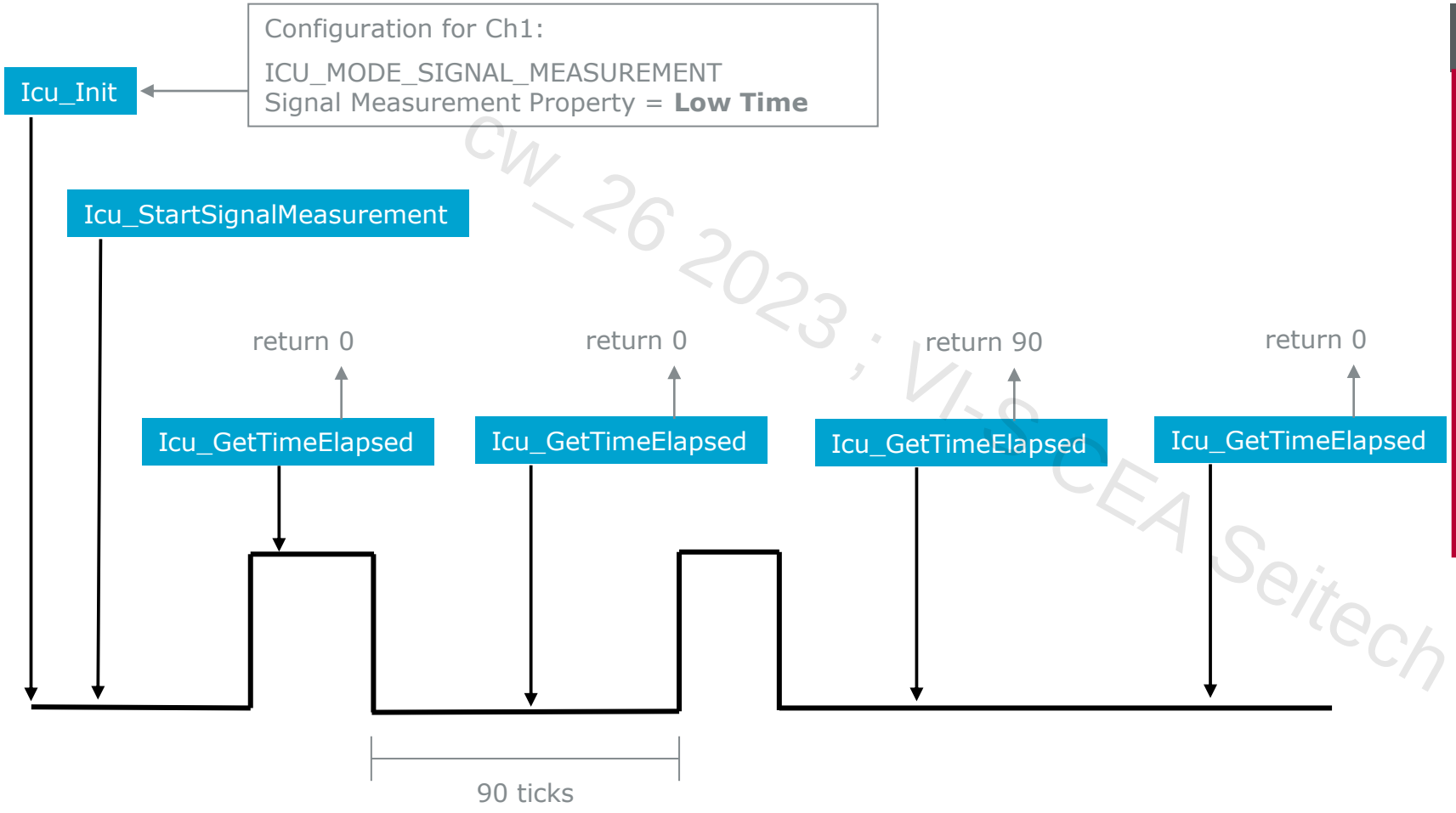
Icu_EnableNotification

Icu_DisableNotification

Callback: Icu_SignalNotification_<Channel>

CW_26 2023 ; VI-S CEA Seitech

Signal Measurement – Example 1



Signal Measurement API

Icu_StartSignalMeasurement

Icu_StopSignalMeasurement

Icu_GetTimeElapsed

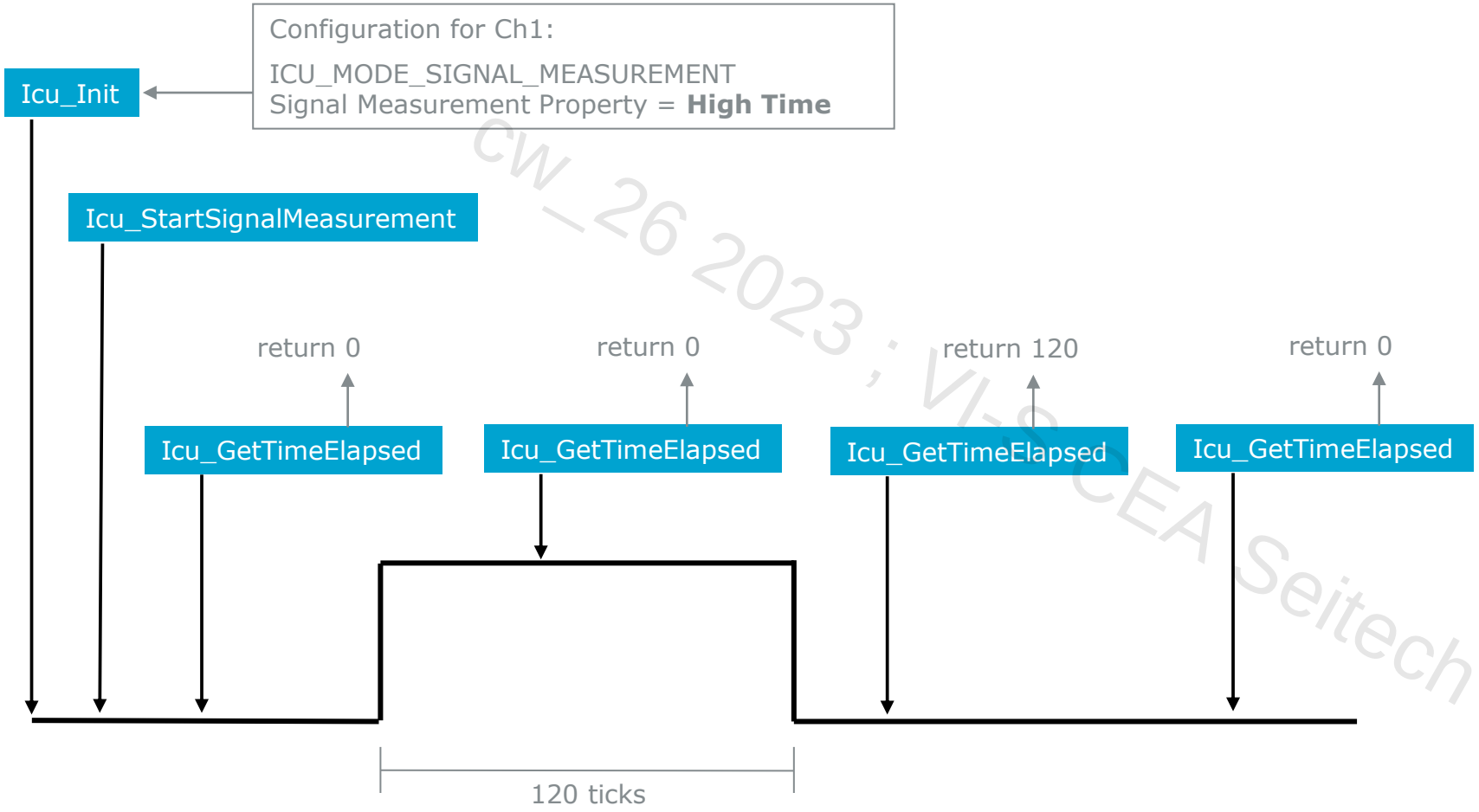
Icu_GetDutyCycleValues

Icu_GetInputState

Measurement Modes: High Time, Low Time, Period Time, Duty Cycle

cw_26 2023 ; VI-S CEA Seitech

Signal Measurement – Example 2



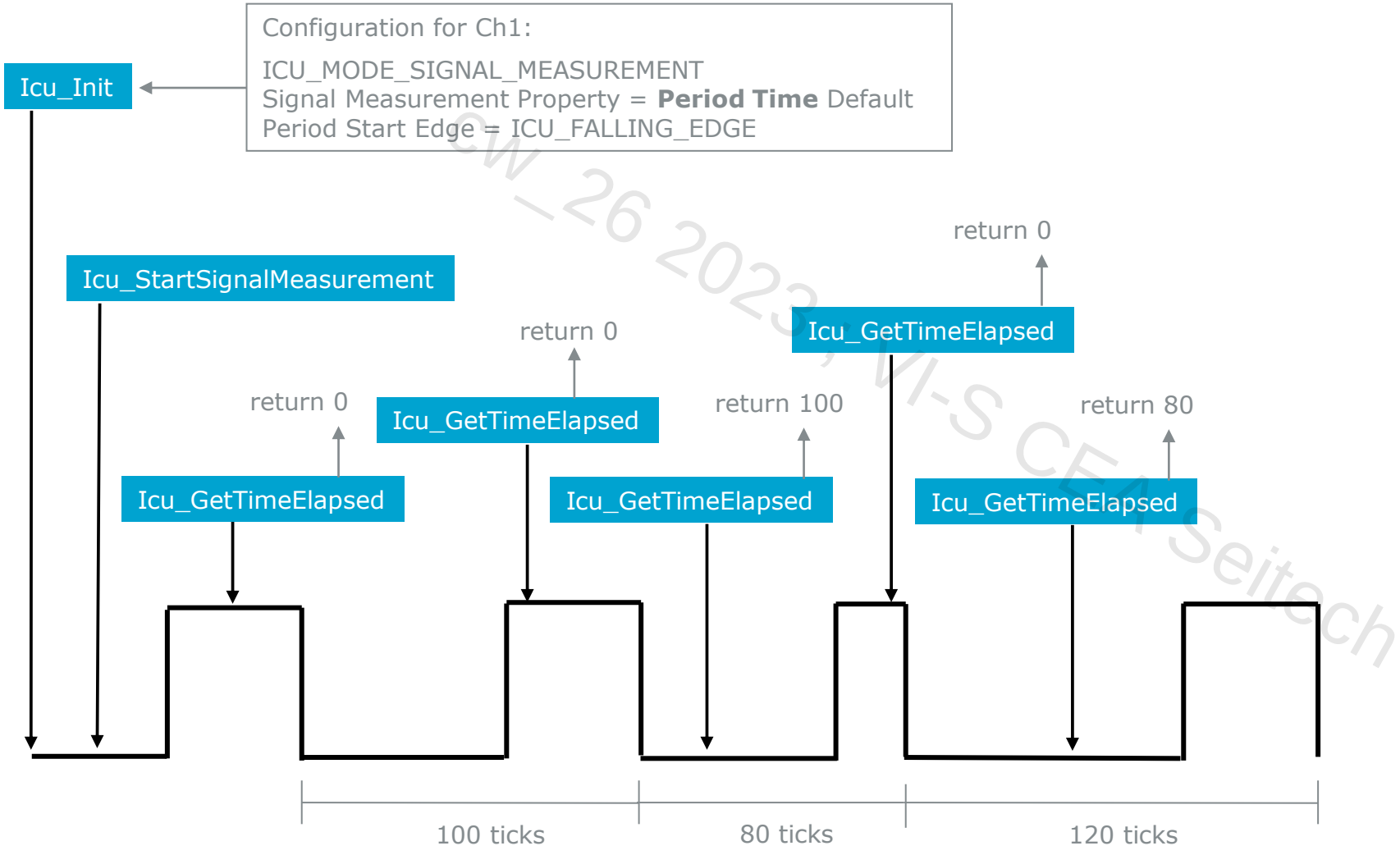
cw_26 2023 ; VI-S CEA Seitech

Signal Measurement – Example 2

1. Icu_Init is called with the configured ConfigPtr
2. Icu_StartSignalMeasurement starts the measurement
3. First call of Icu_GetTimeElapsed returns 0 because no rising edge was detected yet
4. Internal measurement is started with rising edge
5. Second call of Icu_GetTimeElapsed still returns 0 because no falling edge was detected
6. Falling edge ends the measurement of high time
7. Third call of Icu_GetTimeElapsed returns measured high time (120 ticks); Internal measurement value is reset to 0
8. Fourth call of Icu_GetTimeElapsed: same as for 3.

cw. 26 2023 ; VLS CEA Seitech

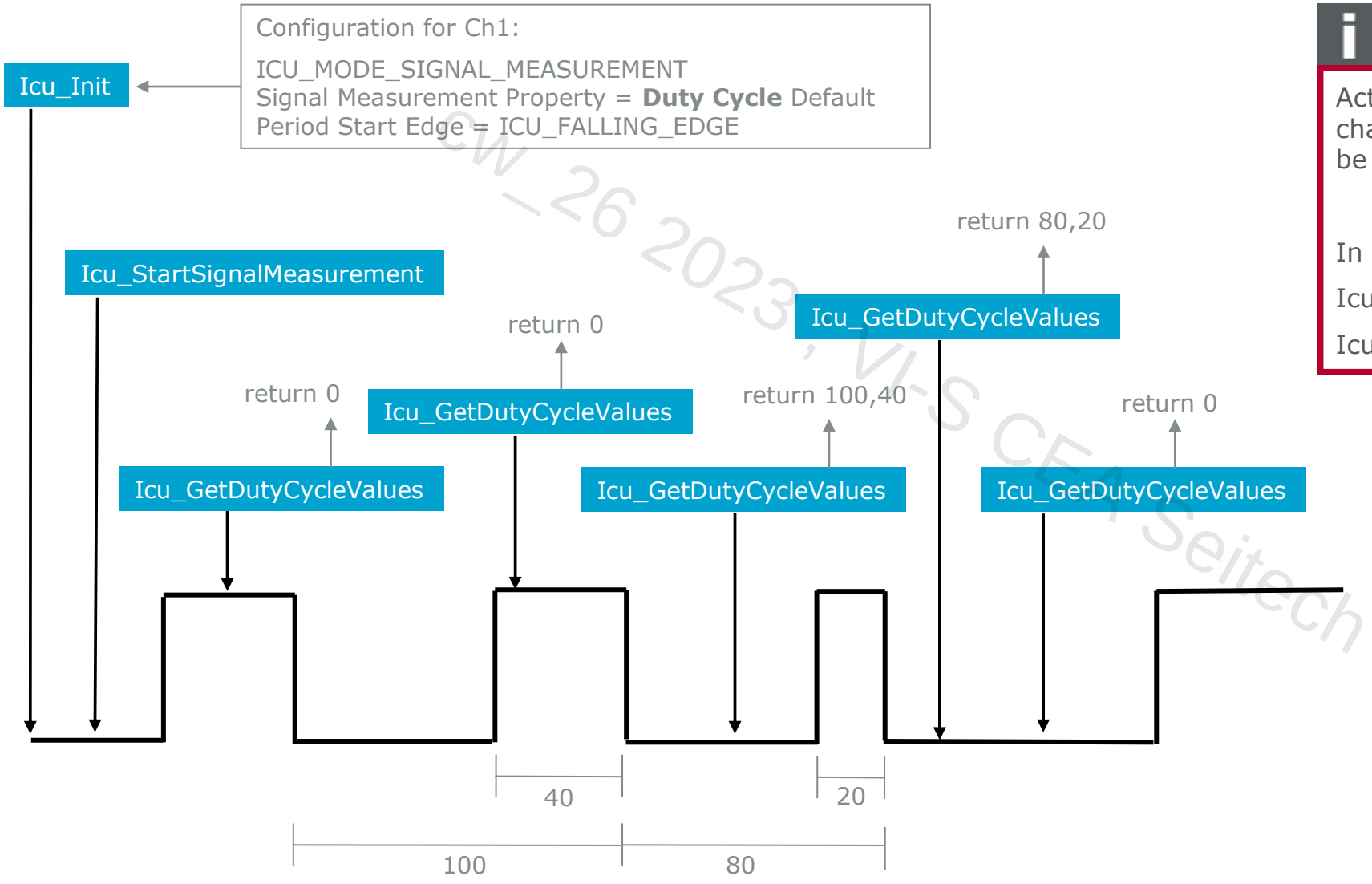
Signal Measurement – Example 3



Activation Condition is not changeable at runtime!!! It needs to be configured at pre-compile time!

cw_26 2023, M-S CE/Seitech

Signal Measurement – Example 4



Activation Condition is not changeable at runtime!!! It needs to be configured at pre-compile time!

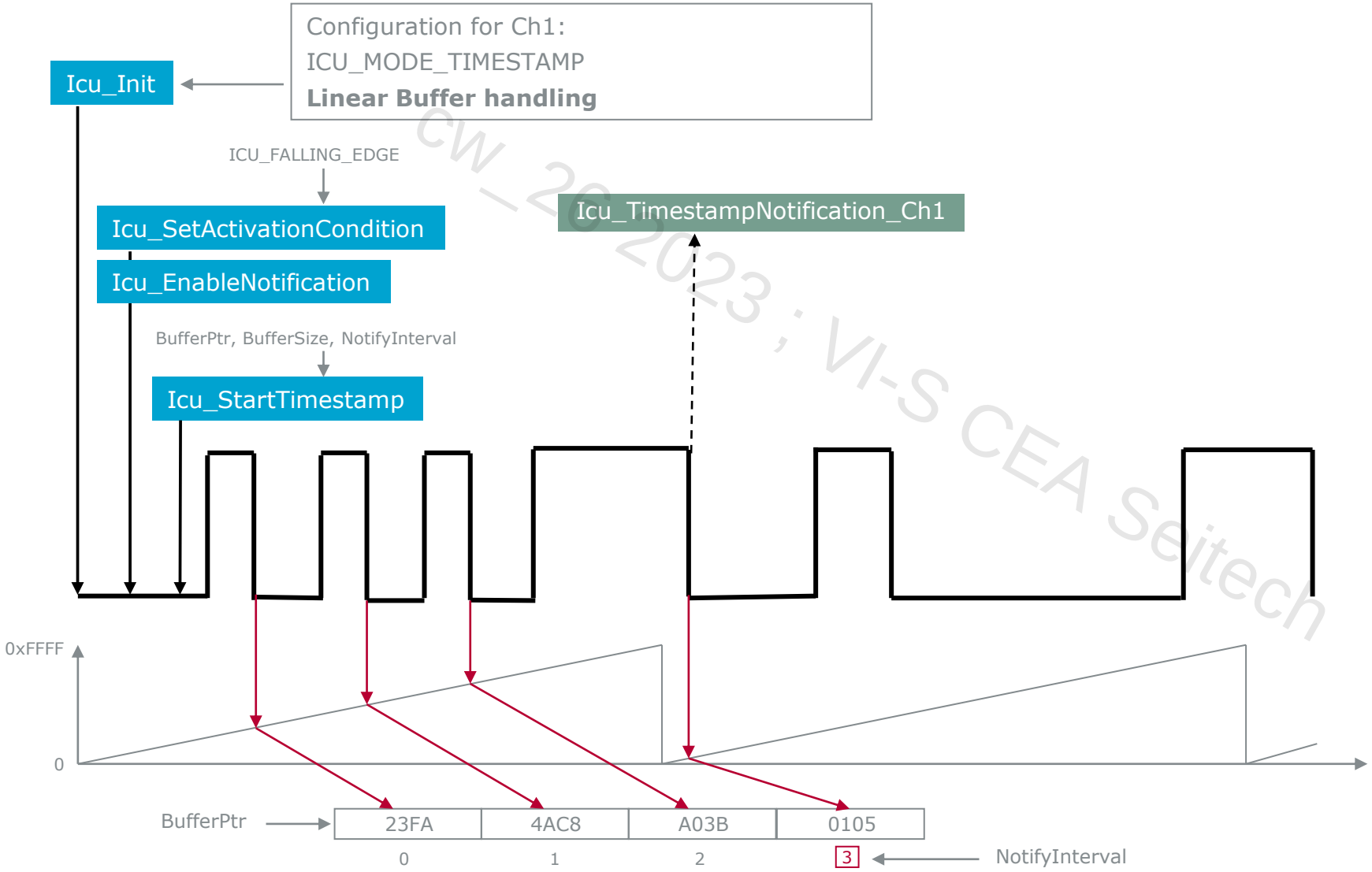
In Example:

Icu_DutyCycleType.ActiveTime = 40

Icu_DutyCycleType.PeriodTime = 100

cw_26 2023, VI-S CEA Seitech

Timestamping – Example 1



cw_26_2023 ; VI-S CEA Seitech

Timestamping – Example 1

Notification when the buffer is **Full**

Timestamping API

Icu_StartTimestamp

Icu_StopTimestamp

Icu_GetTimestampIndex

Icu_SetActivationCondition: ICU_RISING_EDGE, ICU_FALLING_EDGE, ICU_BOTH_EDGES

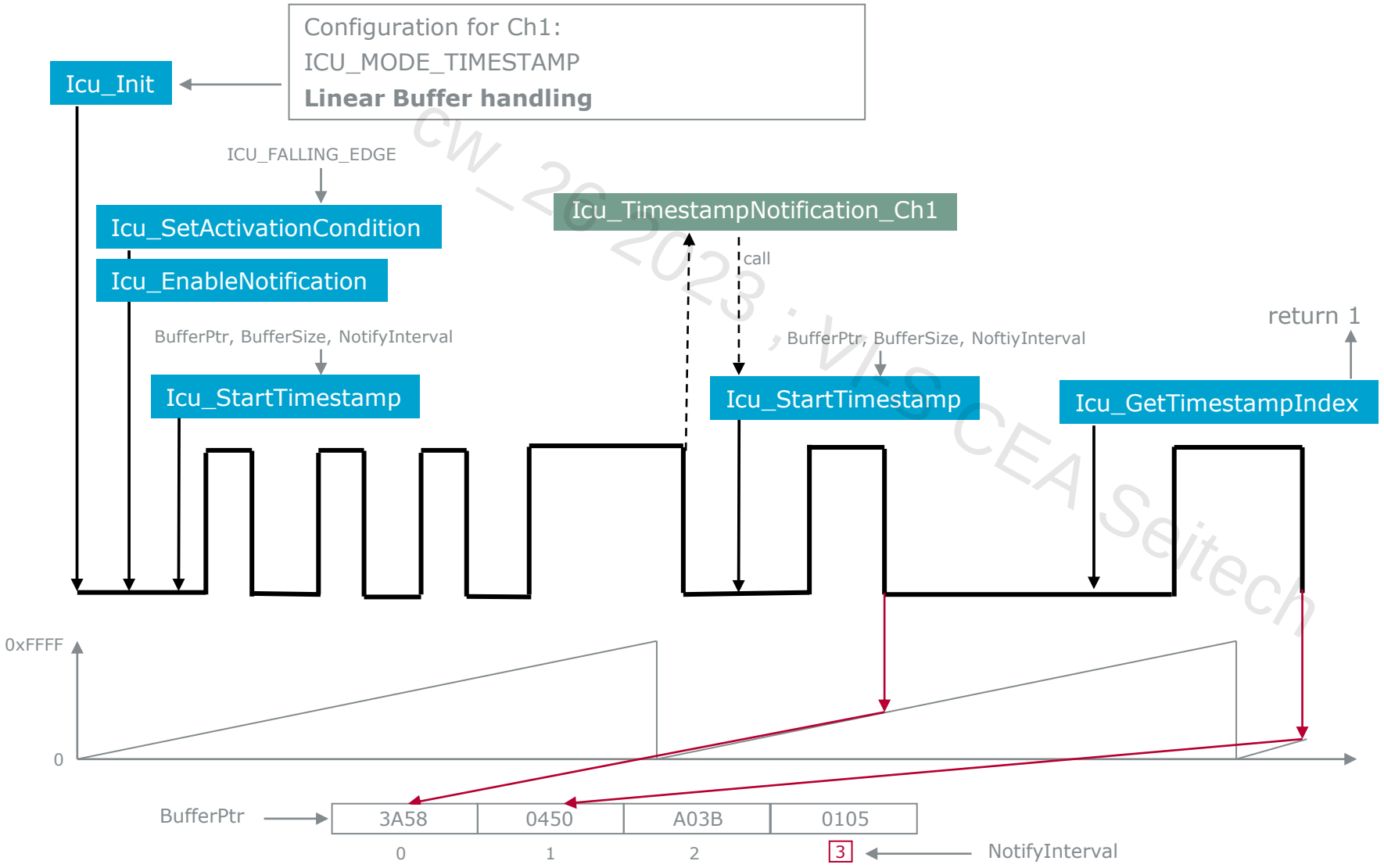
Icu_EnableNotification

Icu_DisableNotification

Callback: Icu_TimestampNotification_<Channel>

cw_26 2023 ; VUS CEA Seitech

Timestamping – Example 1



CW_26_2023 ; VIS CEA Seitech

Timestamping – Example 1

Same as the last page but after the Buffer Full notification, the Index value is retrieved by API

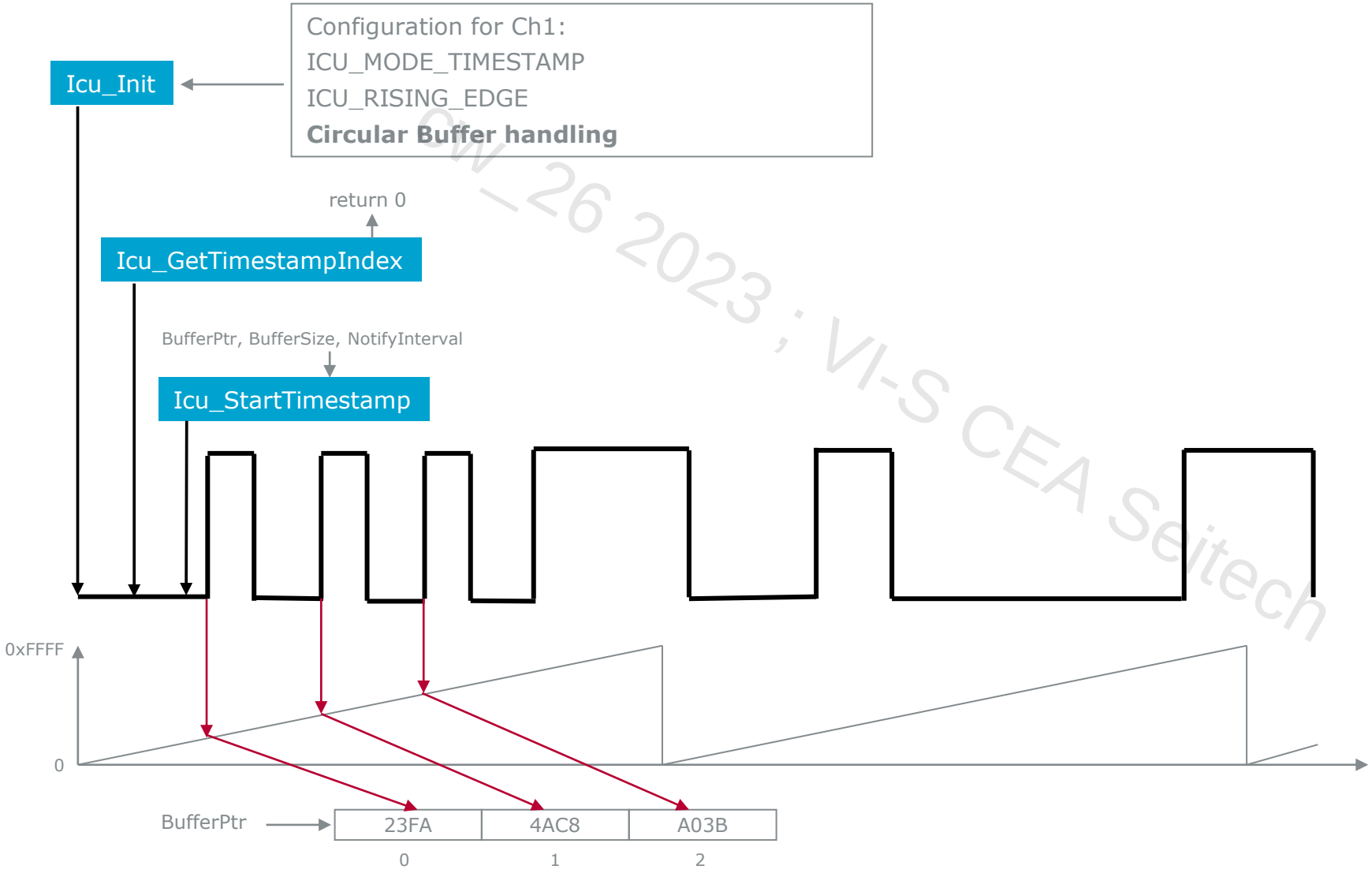
Use a notification function to get informed if buffer is full (or as an alternative poll the TimestampIndex)

Notification function is only called if

1) Notification callback function is configured (it's not NULL_PTR)
2) Notification interval > 0
3) Notification enabled with Icu_EnableNotification
4) Number of events specified by the notification interval is captured

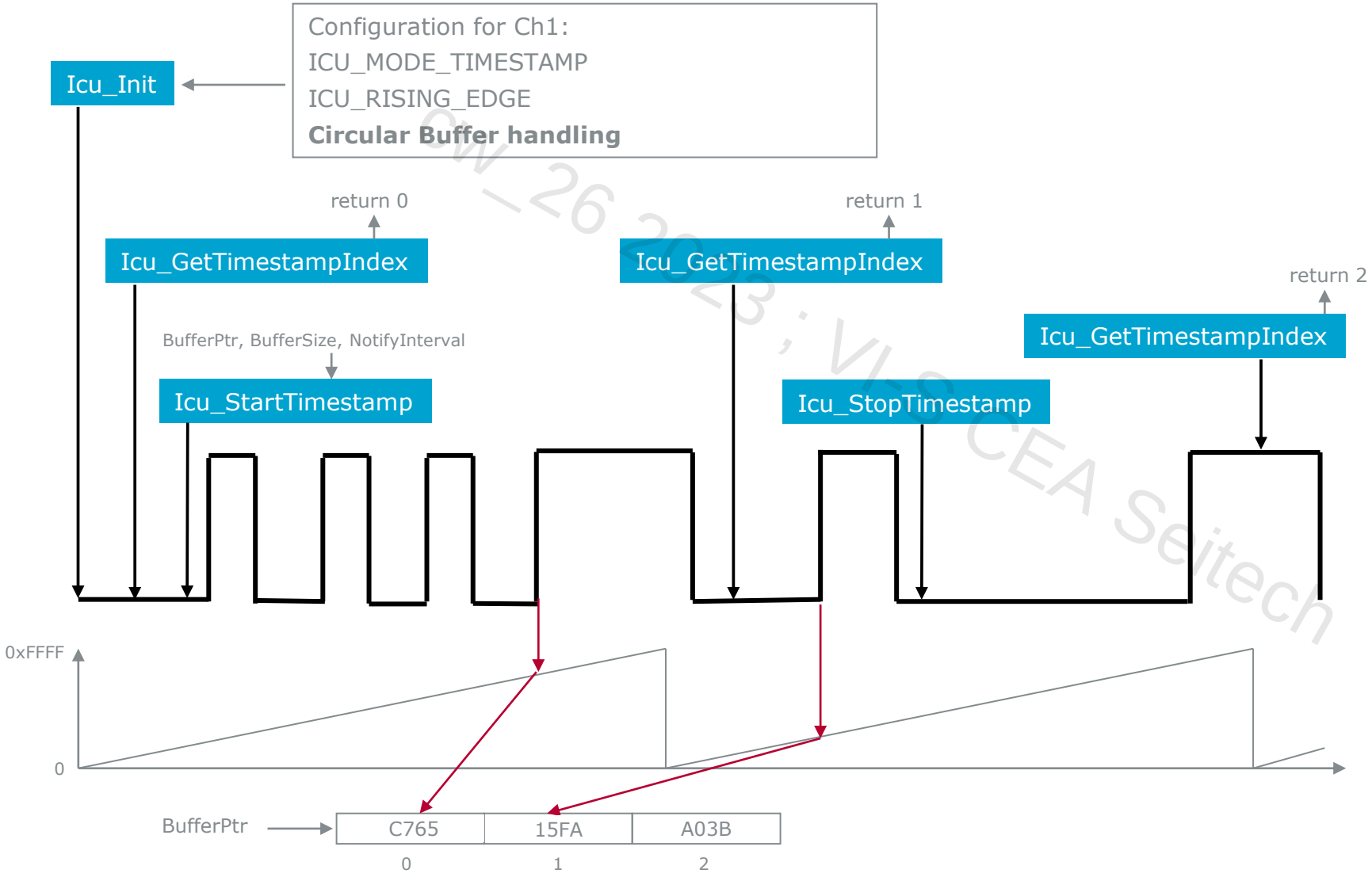
cw_26 2023 ; VI-S CEA Seitech

Timestamping – Example 2



CW_26 2023 ; VI-S CEA Seitech

Timestamping – Example 2

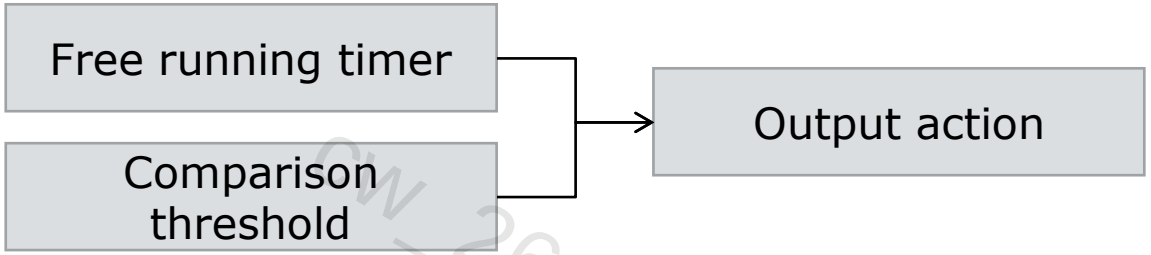


CW_26_2023 ; VLS CEA Seitech

Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- ▶ **OCU Driver (for reference)**
- ADC Driver (for reference)
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech



- Output Compare Unit
  - Start and stop a comparison process
  - Set a comparison threshold
  - Enable and disable notification mechanisms
  - Get counter values
  - Change output pin states
  - Trigger hardware resources (ADC, DMA) if available
- OCU Driver encapsulates
  - Either HW OCU Unit or just a GPT with SW comparison

Here you can define a trigger for some action by other hardware

For example, very special / sophisticated for the Infineon Tricore

CW_26 2023 ; VI-S CEA Seitech

Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- **ADC Driver (for reference)**
- Client Server Concept
- I/O Hardware Abstraction
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Characteristics

- Converts analog signals into digital signals
- Abstracts the access to converted signals
- Different conversion modes configurable
- Notification after conversion configurable

- `Adc_Main_PowerTransitionManager`
  - This API is cyclically called and supervises the power state transitions, checking for the readiness of the module and issuing the callbacks `IoHwAb_Adc_NotifyReadyForPowerState<Mode>`

CW 26 2023 ; VI-S CEA Seitech

Checking if the ADC is powered by the microcontroller:

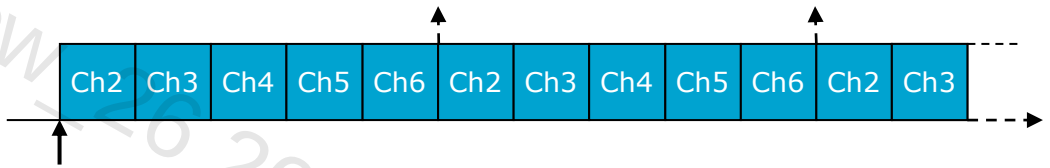
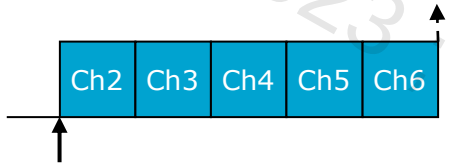
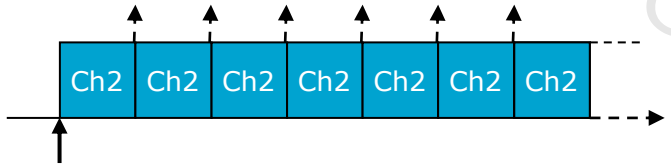
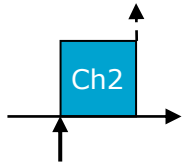
The actual Power State can be handled in the I/O HW Abstraction

Channel groups

- Grouping of ADC channels into ADC channel groups
- Channel Group contains at least one channel
- All channels in a group need to belong to the same ADC HW Unit
- Prioritization of channel groups

CW_26 2023 ; VI-S CEA Seitech

Conversion Modes

Channel Type	Conversion Type	Process
Multi-channel Group	Continuous	
Multi-channel Group	One-Shot	
Single-channel group	Continuous	
Single-channel group	One-Shot	

Start of conversion (hardware or software triggered) Notification (if enabled)

cw 26 2023 VI-S CEA Seitech

Trigger Modes

- Hardware Trigger
  - E.g. expired timer, edge detection on an I/O channel
- Software Trigger
  - Start/Stop of conversion through software API call

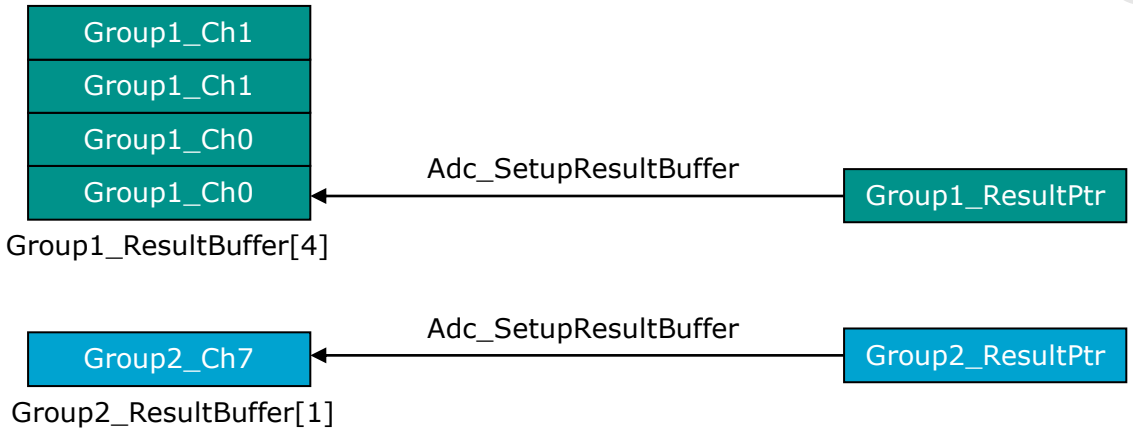
cw_26 2022, VI-S CEA Seitech

```
Adc_EnableHardwareTrigger
Adc_DisableHardwareTrigger

Adc_StartGroupConversion
Adc_StopGroupConversion
```

Buffer initialization

- Result Pointer needs to be initialized during runtime
  - API function: `Adc_SetupResultBuffer`
  - Result Buffer Size = m*n
    - m = Number of channels in group
    - n = Number samples per channel (`ADC_STREAMING_NUM_SAMPLES`)
- ADC driver stores converted results into the Result Buffer



No direct access to ADC hardware registers!

Result pointer (pointer to the result buffer) is a pointer to a memory area specified by the application.

Number of samples for streaming is defined at pre-compile time (Parameter is 1 for Single Access Mode).

Configuration for Group 1:

Group consist of Ch0 and Ch1

Number of samples = 2

Configuration for Group 2:

Group consist of Ch7

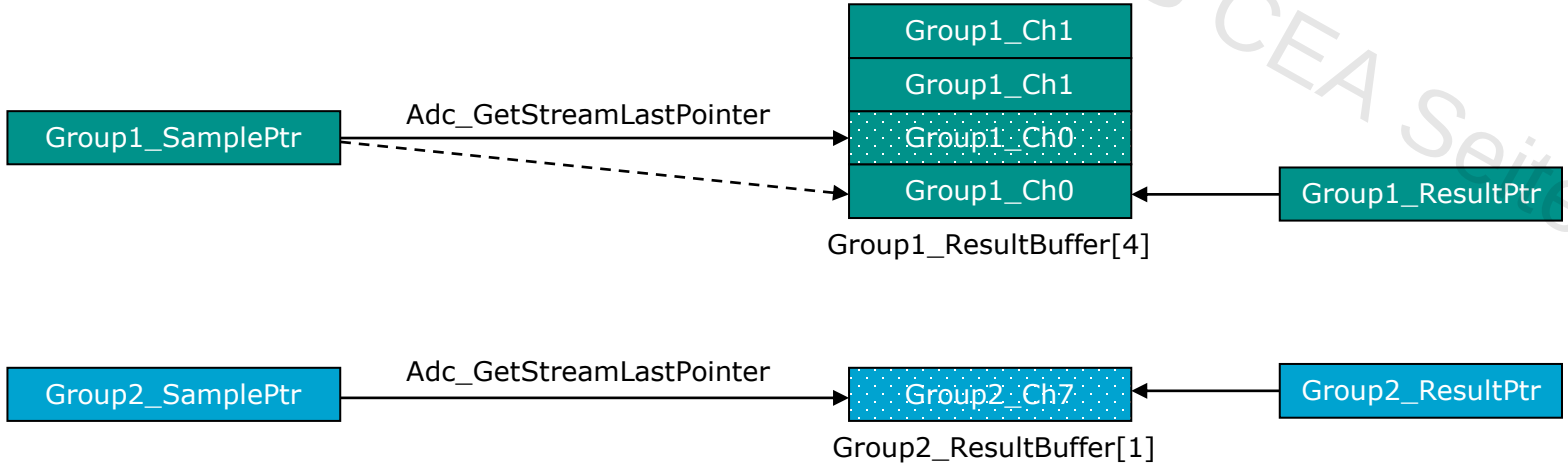
Number of samples = 1 (implicit due to Single Access Mode)

cw_26_2023_VI-S CEA Scilech

Result Access Modes – Streaming Access

- `Adc_GetStreamLastPointer` returns the pointer to the latest converted value of the first group channel and the number of valid samples
- Stream Buffer Mode
  - Circular: Conversion continuous at the beginning of the buffer if the stream buffer is full
  - Linear: Conversion is stopped if buffer is full

cw 20 2023 ; VI-S CEA Seitech



Result Access Modes – Streaming Access

Possible return values of `Adc_GetStreamLastPointer`:

- 0 – No valid value in result buffer
- !0 – Number of valid samples

For accessing further group channels the layout of the result buffer must be taken into account

Configuration for Group 1:

Group consist of Ch0 and Ch1

Number of samples = 2

Configuration for Group 2:

Group consist of Ch7

Number of samples = 1 (implicit due to Single Access Mode)

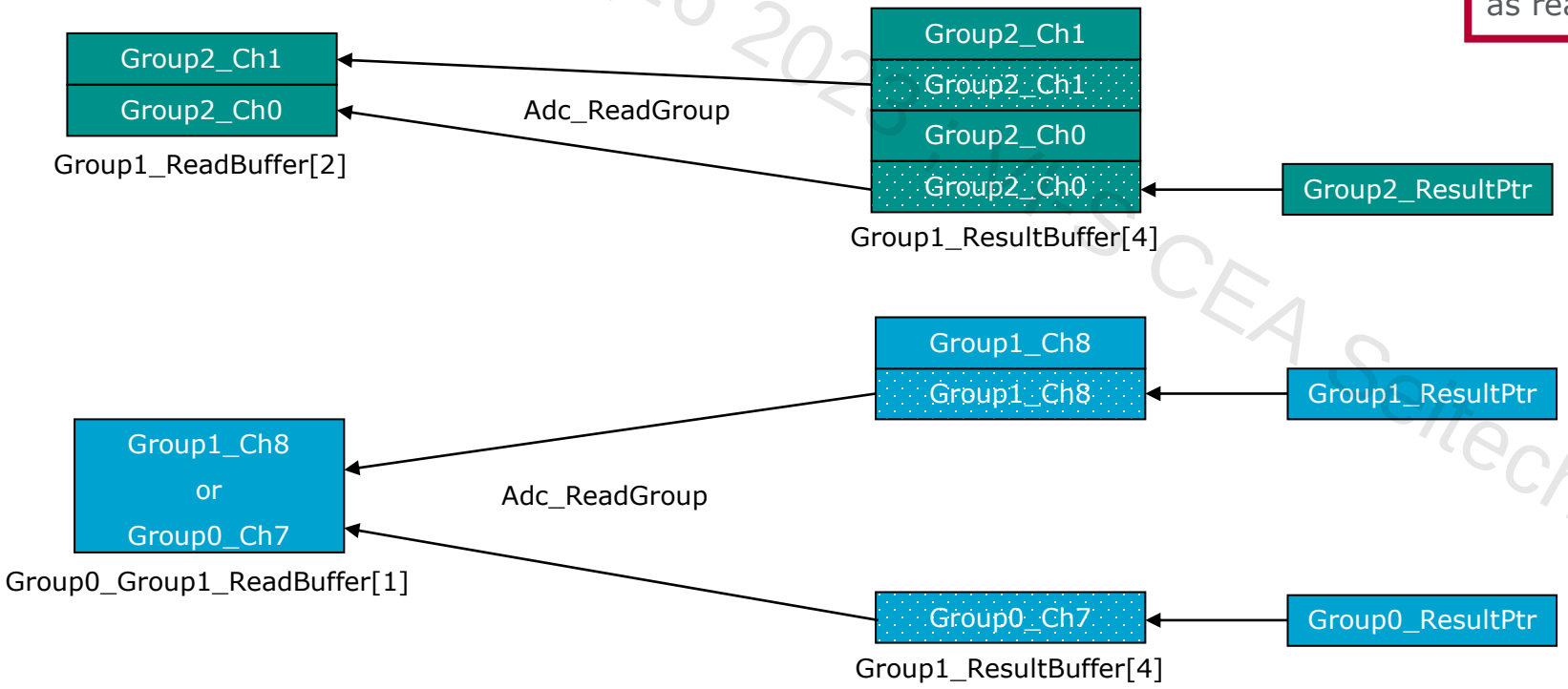
Note: the `Adc_GetStreamLastPointer` API is used for no matter what the access mode is (either Single Access or Streaming Access). In the case of Single Access, we treat it like a Streaming with buffer size 1.

CW_26_2023 ; VI-S CEA Seitech

Result Access Modes – Single Access

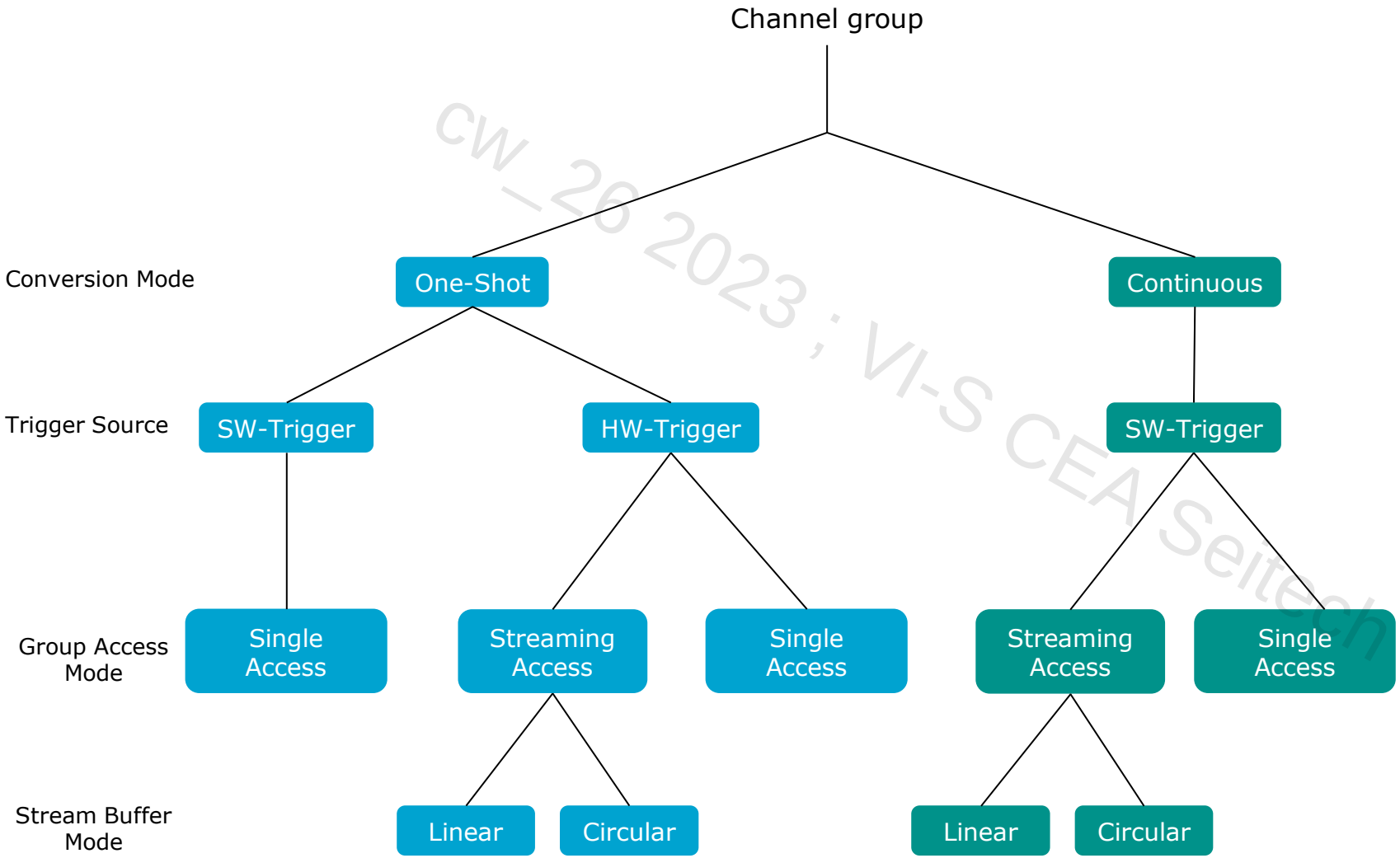
- `Adc_ReadGroup` returns the latest converted values of all channels in a group

Group0_Group1_ReadBuffer[1] is used as read buffer for Group 0 and Group 1



cw_26 2025 © CEA Sentech

Channel group configuration options



cw_26 2023 ; VI-S CEA Seitech

Channel group configuration options

Pre-Compile Configuration:

Adc_GroupConvModeType: ADC_CONV_MODE_ONESHOT, ADC_CONV_MODE_CONTINUOUS

Adc_TriggerSourceType: ADC_TRIGG_SRC_SW, ADC_TRIGG_SRC_HW

Adc_GroupAccessModeType: ADC_ACCESS_MODE_SINGLE, ADC_ACCESS_MODE_STREAMING

Adc_StreamBufferModeType: ADC_STREAM_BUFFER_LINEAR, ADC_STREAM_CIRCULAR

If Hardware Trigger is used:

Adc_HwTriggerSignalType: ADC_HW_TRIG_RISING_EDGE, ADC_HW_TRIG_FALLING_EDGE; ADC_HW_TRIG_BOTH_EDGE

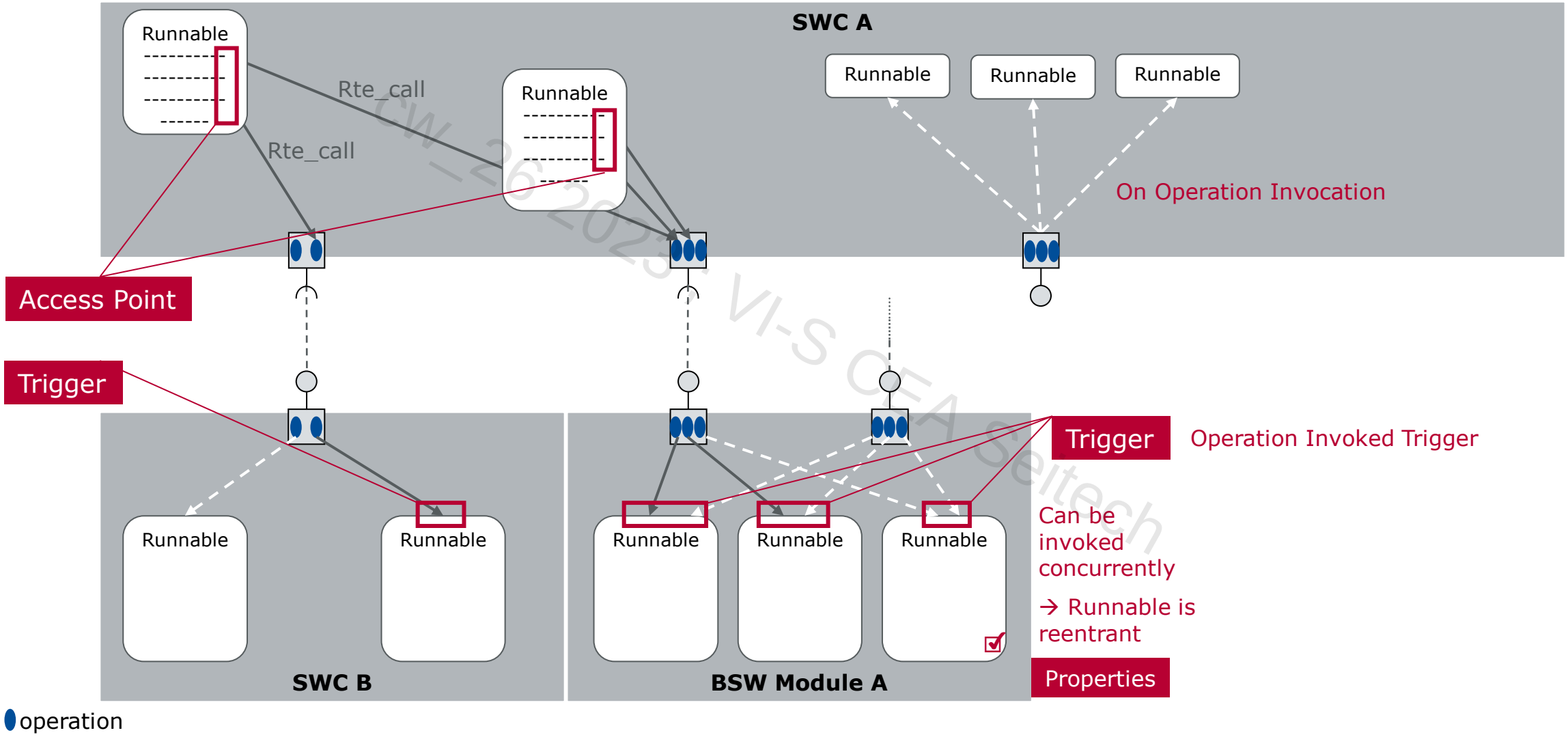
cw_20_2023; VI-S CEA Seitech

Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- ▶ **Client Server Concept**
- I/O Hardware Abstraction
- Coming up next

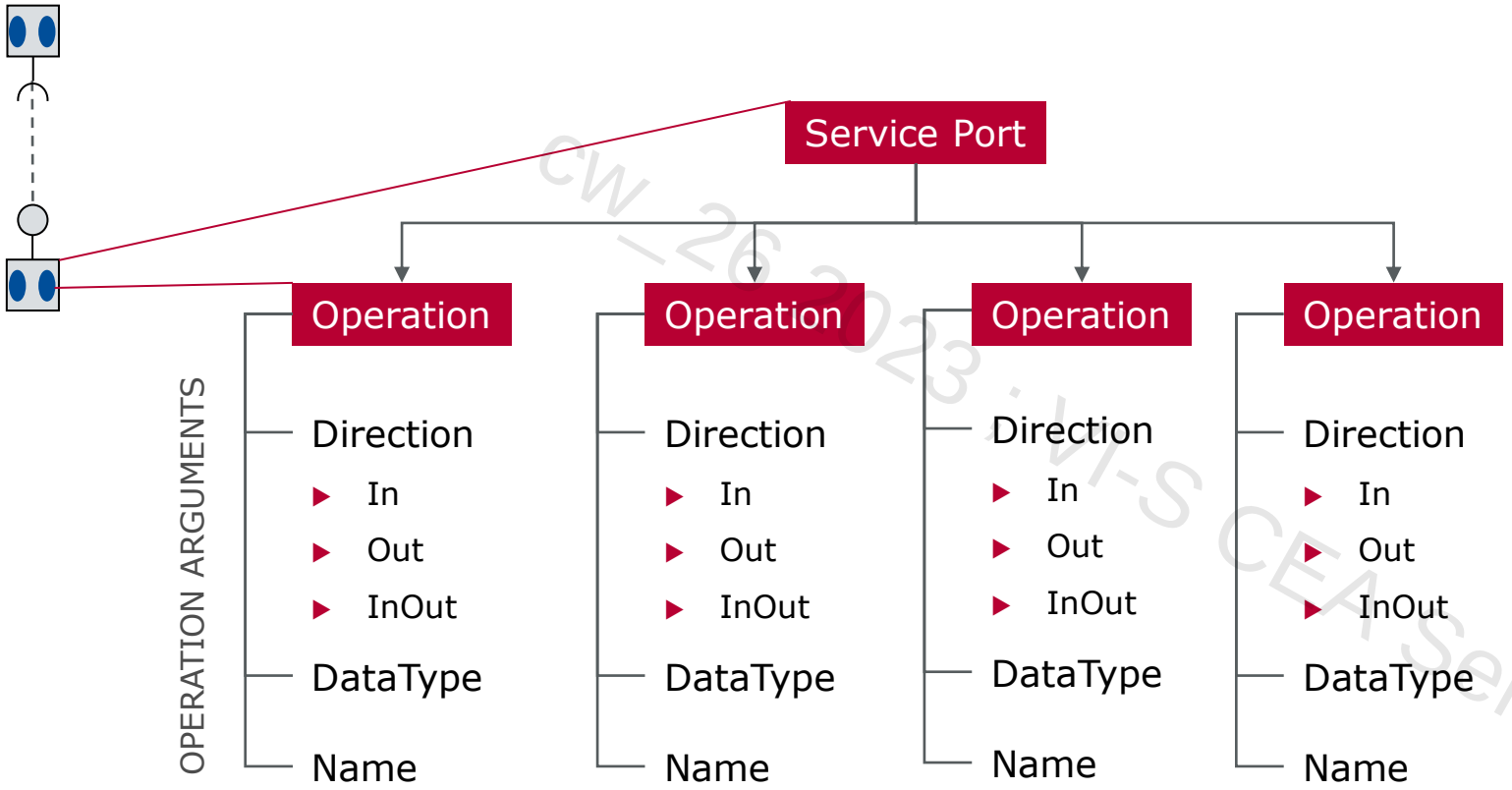
CW_26 2023 ; VI-S CEA Seitech

Call of Server Ports



CW_26_2023_VI-S CEA Seitech

Operations



cw_26_2023 · VI-S CEA Seitech

A Server has to provide one Runnable per Operation

VECTOR

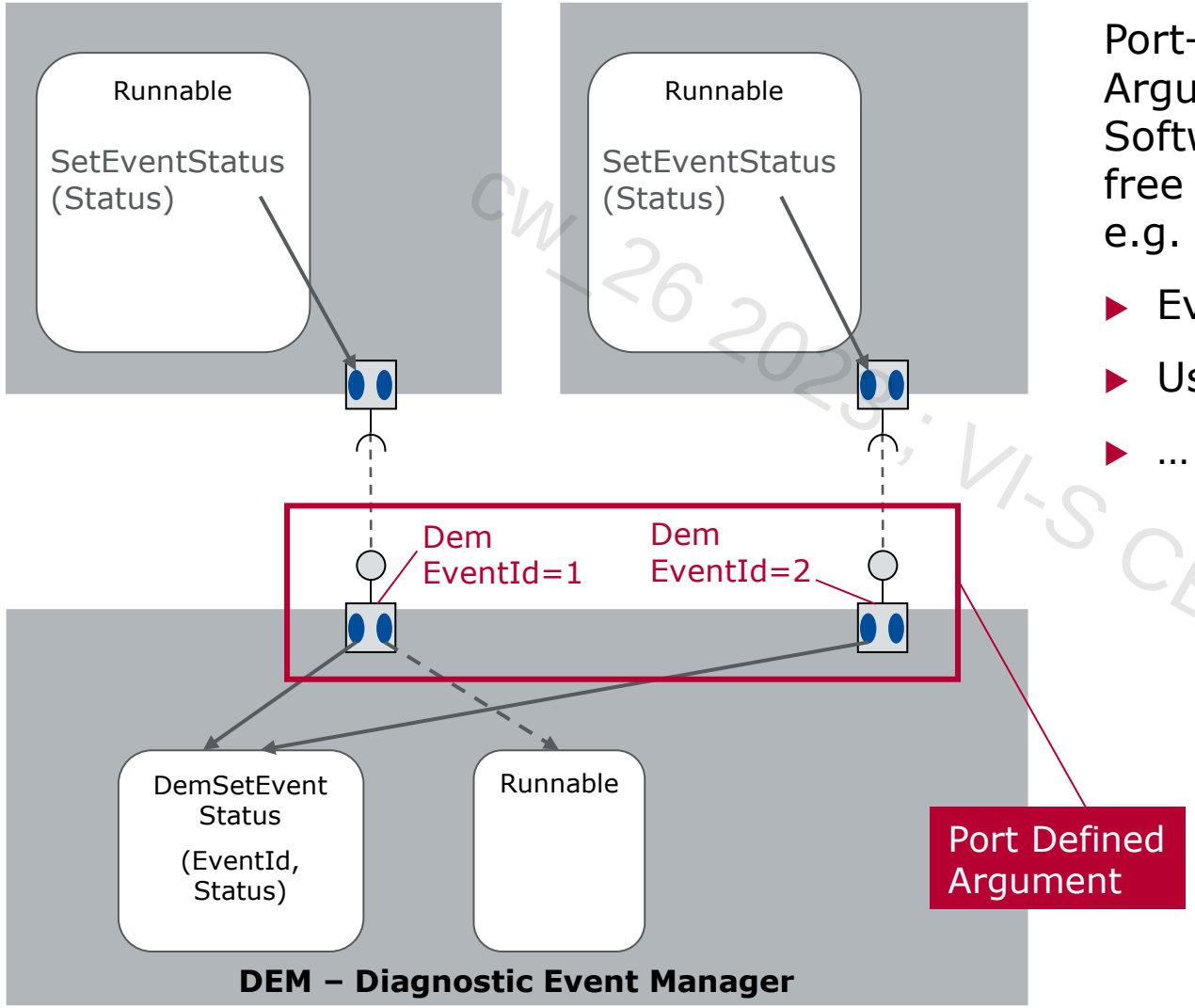
Operations

Arguments IN, IN/OUT, OUT

- IN
  - By value or by reference according to IDT
- OUT
  - By reference
- IN/OUT
  - Synchronous: By reference
  - Asynchronous: IN/OUT degrades to IN → by value
  - All pointers must be valid until API call returns

CW_26 2023 ; VL-S CEA Seitech

Port-Defined Argument



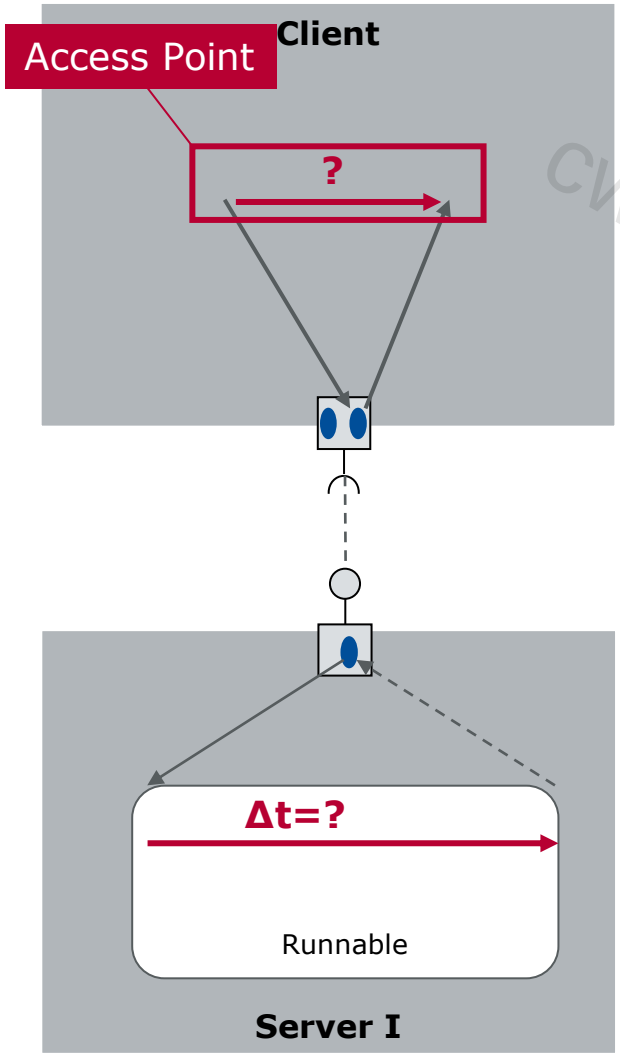
operation

Port-Defined Arguments keep Software Components free of dependencies e.g. of

- EventIds (DEM)
- Users (ComM)
- …

cw_26 2025 ; VI-S CEA Seitech

Overview: Runtime of Servers



How to wait for the answer of a service (Runnable)?

- Synchronous → Timeout [unit]
- Asynchronous → Waiting → Timeout [unit]
- Asynchronous → Polling
- Asynchronous → None (trigger Runnable)

Get result using:

RequestResult on:

- Task Level
- In a triggered Runnable (if possible)

cw_26 2023 : VI-S CEA Seitech

Synchronous with / without Timeout

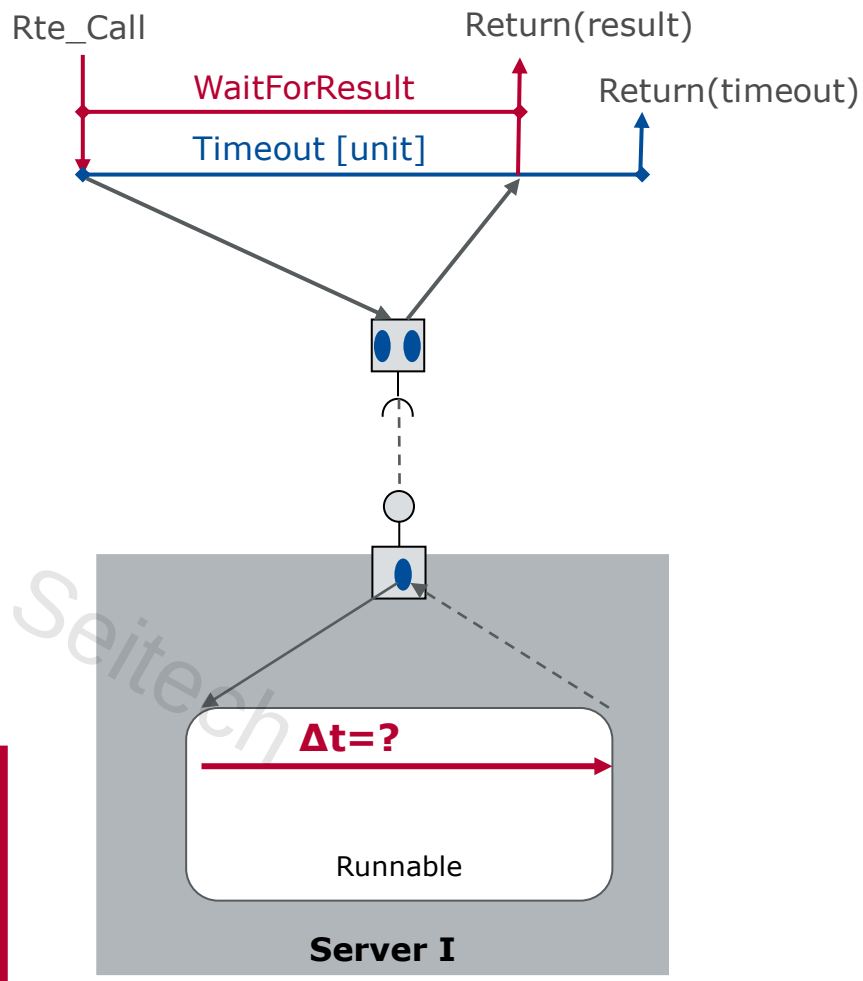
Synchronous → Timeout [unit]

The Rte_Call

- Triggers the timeout monitoring
- Waits for the result

The return value contains:

- The result of the server when return occurs before the timeout expires
- An timeout error code as the timeout expires before the result of the server is ready



"With Timeout" requires a Task mapping (in the Operating System configuration)

→ Before activating the other task an OS alarm is started

→ When the alarm expired = TIMEOUT

Task Mapping can be configured:

-Consumes more runtime

cw 26 2023 ; VI-S CEA Seitech

Asynchronous with Timeout

Asynchronous → Waiting → Timeout [unit]

The Rte_Call

- Triggers the server (Runnable) and returns immediately

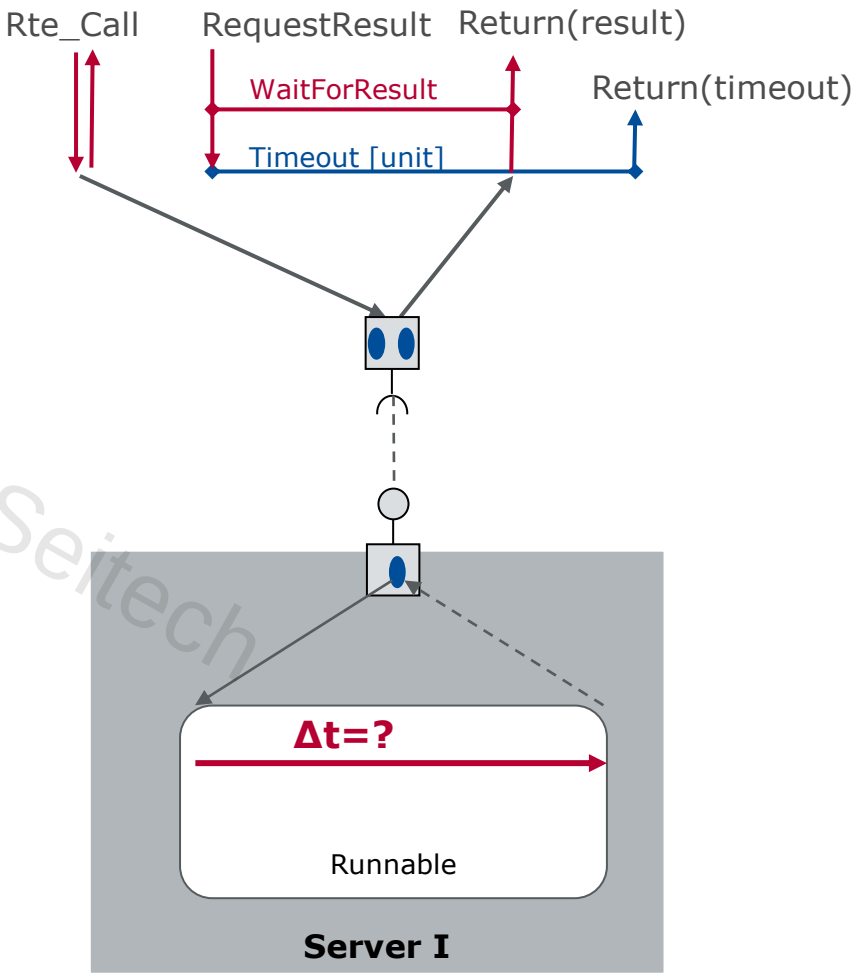
The Rte_Result

- Triggers the timeout monitoring
- Waits for the result

The return value contains:

- The result of the server when return occurs before the timeout expires
- A timeout error code if the timeout expires before the result of the server is ready

CW_26 2023 ; VI-S CEA Seitech



1. Rte_Call triggers an RTE / OS event
2. Rte_Result (waiting) will activate an alarm for timeout monitoring:
   AND call WaitEvent (waiting point until the Server runnable has finished)
3. Server runnable has finished and sets the event for the RTE_RESULT

Asynchronous and Polling

Asynchronous → Polling

The Rte_Call

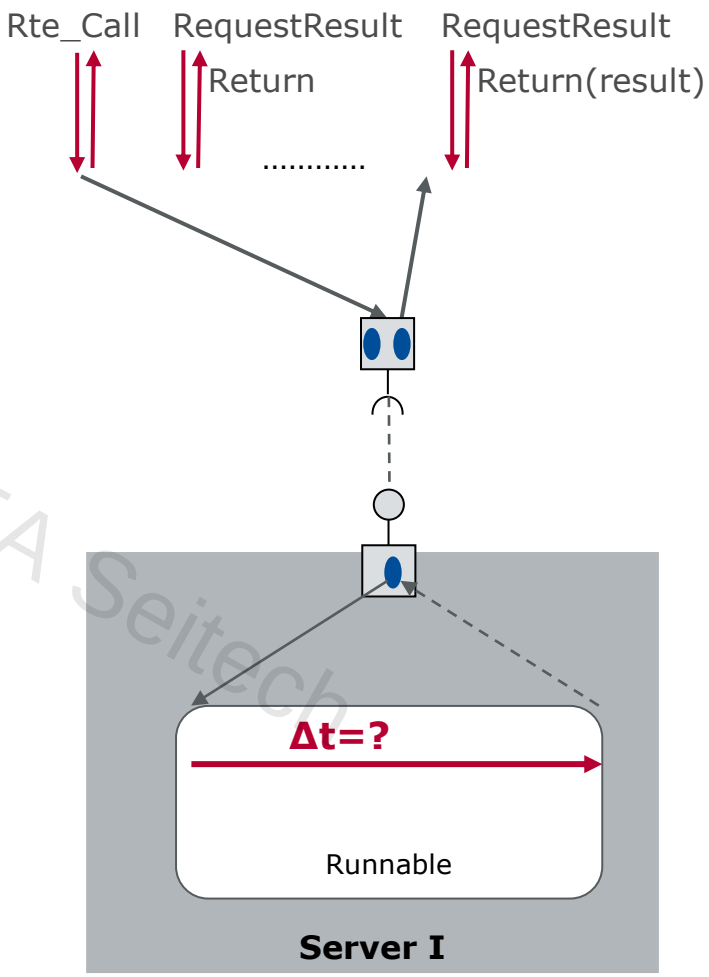
- Triggers the server (Runnable) and returns immediately

The Rte_Result

- Sees if the result is already there

The return value contains:

- The result of the server if present
- Information that result is not ready yet



cw 20 2023 ; VI-S CEA Seitech

Asynchronous and Trigger on Runnable

Asynchronous → None

The Rte_Call

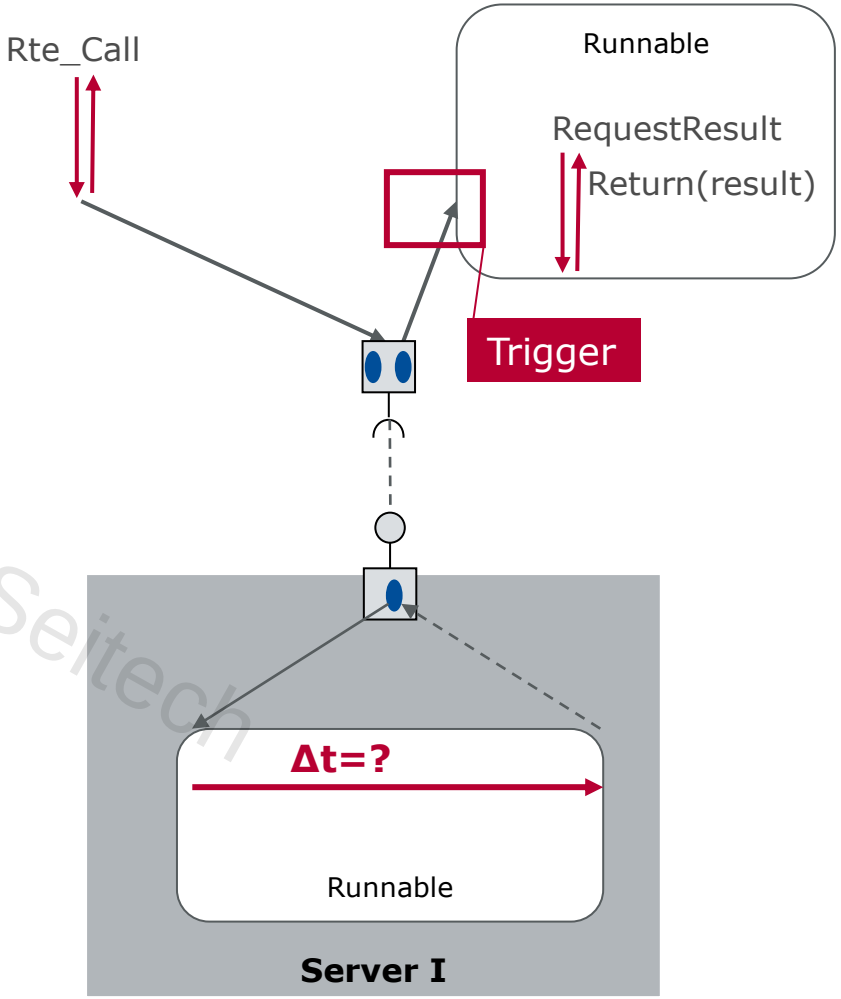
- Triggers the server (Runnable) and returns immediately

The Result is present

- The assigned Runnable is triggered

Get result

- As the result triggers the Runnable, the result is now present
- To get the result a an Rte_Result is necessary in the triggered Runnable



1. Rte_Call sets an Event for the Server Runnable
2. When the Server Runnable has finished, another Event will be set to notify the configured client Runnable
3. Everything should be configured already through the RTE Runnable Triggers

cw 20 2023 ; VI-S CEA Seitech

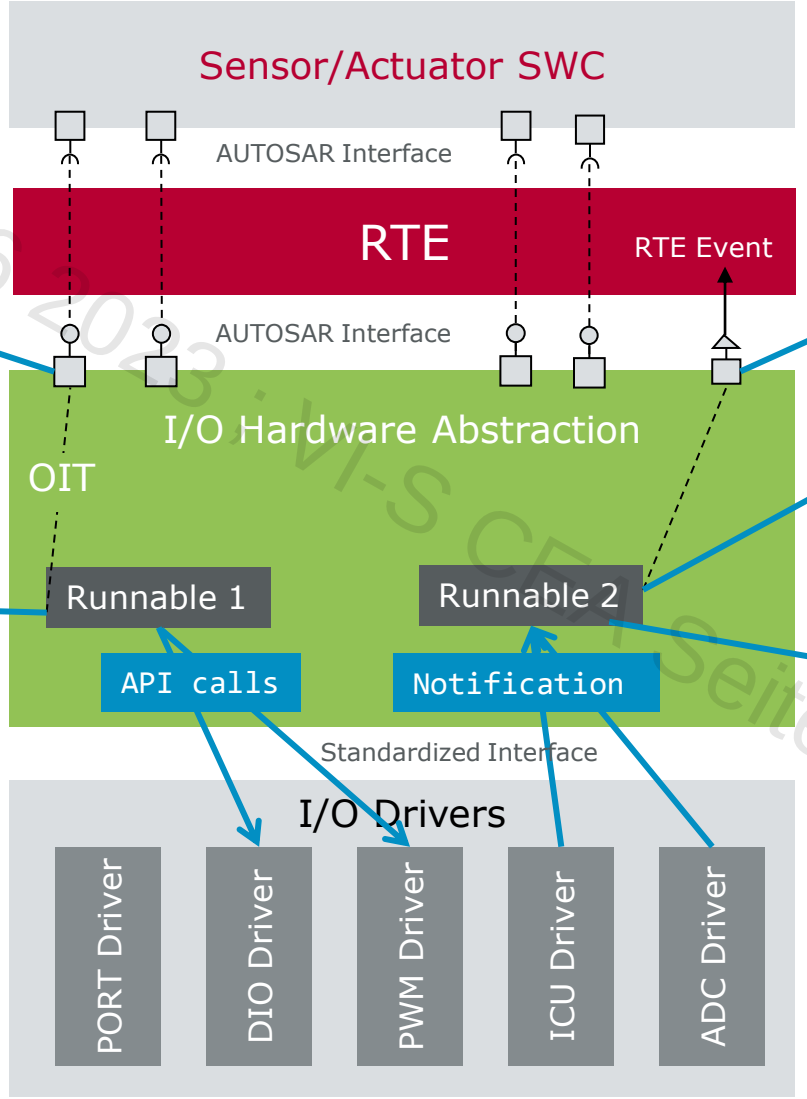
Agenda

- Architecture of I/O
- I/O Drivers – General Characteristics
- PORT Driver
- DIO Driver
- PWM Driver (for reference)
- ICU Driver (for reference)
- OCU Driver (for reference)
- ADC Driver (for reference)
- Client Server Concept
- ▶ **I/O Hardware Abstraction**
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Implementation of the IO HwAb

Client/Server Operations e.g.
- OP_GET
- OP_SET
- OP_TEST
- OP_DIAG
- OP_REPORT



Sender/Receiver Communication (e.g. to trigger Runnable)

Access Point

Operation Invoked Trigger, [task mapping]

No Trigger, binding over Implementation (enter Runnable Name as Notification function in driver), no task mapping possible; ISR context

Notifications are called in interrupt context.

Only OP_GET, OP_SET are supported in Vector Tools

No Sender/Receiver communication supported in Vector Tools

IoHwAb can be equivalently implemented as a Complex Driver (CDD)

cw_26_2023 vi-S CE Seitech

ECU Degradation concept

- Power State Transitions
  - Map ECU Power Modes to peripheral Power Modes over respective Power State API
- I/O HW Abstraction (IoHwAb) shall be able to
  1. Prepare the switch into certain Power State
  2. Execute transitions into the desired Target ECU Power State
  - This separation shall render it useless to poll for allowed transitions.
- Examples for modules supporting this concept:
  - ADC
  - PWM

CW_26_2023 ; VI-S CEA Seitech

I/O

Exercise: I/O

cw_26 2023, VI-S CEA Seitech